



Mathematical Problem-Solving Workflow

Full-fidelity process protocol for `pid-rs`

`pid-rs` contributors

4 August 2026

Abstract

This document is the human-readable typeset rendition of the canonical Markdown file `MATHEMATICAL_PROBLEM_SOLVING_WORKFLOW.md`. It records external process observations, then defines the complete project protocol for exact claims, independent routes, retained failures, certificate conversion, adversarial audit, blind holdouts, PID-specific exceptional cases, Codex goal and continuity control, a future typed publication harness, layered assurance, and claim disposition. The protocol is project-defined guidance. It is not a PID estimator, theorem, benchmark result, or validation claim; the harness described here is a specification, not an implemented system.

Contents

1	Reader's primer: objects, logic, and evidence boundaries	3
1.1	Status of this primer	3
1.2	Four objects that must not be conflated	3
1.3	Exact claims, quantifiers, and counterexamples	4
1.4	Certificates and independently checkable computation	5
1.5	PID vocabulary and a finite worked reconstruction	5
1.6	Negative example: an unseen rare cell	6
1.7	Negative example: pairwise is not mutual independence	6
1.8	Bounded evolutionary search: useful scope and hard boundary	6
2	Typed assurance and rigorous evidence aggregation	7
2.1	Status of this supplement	7
2.2	Preserve a vector across assurance layers	7
2.3	Critical cut sets determine dependence	8
2.4	Worked example: five correlated test families are one confirmation at a shared oracle	8
2.5	When a model-conditional posterior belief is permitted	9
2.6	Change invalidation and publication state	10
3	Mathematical problem-solving workflow for <code>pid-rs</code>	11
3.1	Status and scope	11
3.2	Compact glossary	11
3.3	Primary-source pins for PID vocabulary and routing	14
3.4	Source observations	14
3.4.1	X thread	14





3.4.2	Cycle Double Cover prompt	15
3.4.3	Erdős repository	16
3.4.4	Erdős 477 cubic repository	16
3.4.5	Corrected vector-bundle claim: a typed citation-edge failure	17
3.4.6	External AI discussions	19
3.4.7	OpenAI-hosted proof-process walkthroughs: process controls only	19
3.5	AI model operating protocol	22
3.5.1	Applicability and risk	22
3.5.2	Separate roles	22
3.5.3	Independence vector	22
3.5.4	Independent routes	23
3.5.5	Critical-cut-set accounting	24
3.5.6	Frozen run context	24
3.5.7	Freeze purpose and identity boundary	25
3.5.8	Durable agent continuity	26
3.5.9	Codex goal, plan, and tool lifecycle	27
3.5.10	Tool-call and agent receipts	28
3.5.11	Publication-harness state machine	29
3.5.12	Evidence labels and route memos	30
3.6	pid-rs protocol	31
3.6.1	1. Write an exact-claim packet	31
3.6.2	2. Build an obligation graph	33
3.6.3	3. Keep a separate-approach registry	34
3.6.4	4. Retain blockers and failed routes	34
3.6.5	Load-bearing correction ledger	35
3.6.6	5. Convert computation into certificates	35
3.6.7	6. Run the required 20-lens adversarial audit	36
3.6.7.1	Lenses 1–10: scientific object and inference contract	36
3.6.7.2	Lenses 11–20: evidence, custody, implementation, and release contract	36
3.6.7.3	SxPID definition-compatibility firewall	37
3.6.7.4	Imported-theorem application map	39
3.6.7.5	Citation-edge type check	39
3.6.7.6	Cheap invariant probes	41
3.6.8	7. Use a blind holdout protocol	41
3.6.8.1	Minimum blind-benchmark commitment	42
3.6.9	8. Maintain a claim-to-evidence matrix	43
3.7	Application to shared-exclusions PID	44
3.7.1	Exact claim packet for a new PID theorem	44
3.7.2	PID routing checkpoints	44
3.7.3	Complementary PID audit families	46
3.7.4	Required counterexample search	47
3.7.5	Formal and software evidence	47
3.7.6	Statistical and ecosystem evidence	48
3.8	Full semantic closure	48
3.9	PID exceptional-case checklist	49
3.10	Layered assurance and go/no-go gates	50
3.11	Acceptance rule	50





1 Reader's primer: objects, logic, and evidence boundaries

1.1 Status of this primer

This section is a typeset-only explanatory expansion. It introduces the vocabulary and works through examples needed to apply the protocol without assuming prior knowledge of formal methods, statistics, or partial information decomposition (PID). It does not change the canonical protocol reproduced in full beginning with Section 3. Every example below is project-defined process guidance, not a new PID theorem or estimator result.

1.2 Four objects that must not be conflated

Let P denote a population law and let $F(P)$ denote a mathematical information functional. Given rows $Z_1, \dots, Z_n$, an estimator is an algorithmic rule

$$\hat{F}_n = A_n(Z_1, \dots, Z_n).$$

A particular program returns a represented number

$$\tilde{F}_n = \text{Program}_{\text{compiler, features, platform}}(Z_1, \dots, Z_n),$$

and a consumer may use that number, diagnostics U_n , and a policy $\mathcal{D}$ to make a decision

$$d_n = \mathcal{D}(\tilde{F}_n, U_n).$$

These are four different objects: the population functional, the estimator, the executable result, and the consumer decision. A proof that F is continuous establishes no finite-sample error rate for $\hat{F}_n$. An estimator theorem does not prove that the executable implements the estimator. Executable conformance does not establish that a downstream decision is safe. The assurance path therefore contains three distinct transitions, each with its own obligation:

$$\boxed{\text{definition}} \rightarrow \boxed{\text{estimator theorem}} \rightarrow \boxed{\text{executable conformance}} \rightarrow \boxed{\text{consumer qualification}}.$$

Each arrow is a separate obligation. Evidence for a box cannot silently replace evidence for an arrow.

ASSURANCE ARCHITECTURE

Keep the scientific objects distinct

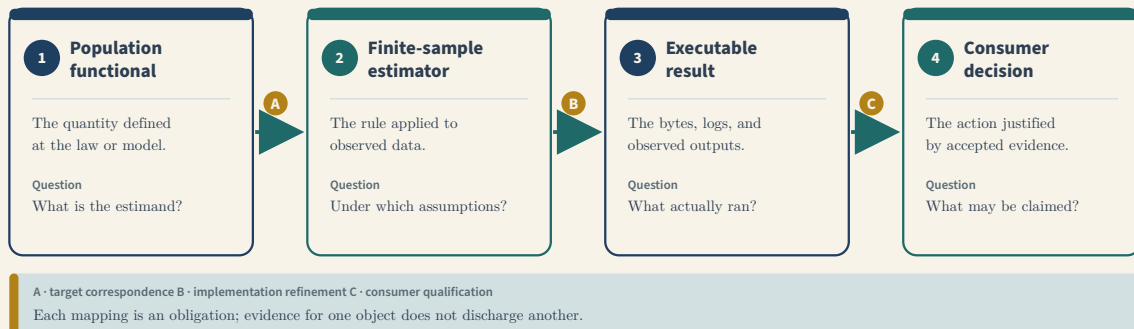


Figure 1: The assurance chain has four distinct objects and three separately justified assurance transitions.





1.3 Exact claims, quantifiers, and counterexamples

An exact mathematical claim should be reducible to a proposition such as

$$\forall x \in \mathcal{X}, \quad A_1(x) \wedge \dots \wedge A_k(x) \implies C(x), \quad (1)$$

where $\mathcal{X}$ is the domain, A_i are assumptions, and C is the conclusion. A counterexample is one explicit $x_\star \in \mathcal{X}$ for which every $A_i(x_\star)$ is true but $C(x_\star)$ is false. An example outside $\mathcal{X}$, or one that violates an assumption, does not refute (1); it may instead show that the assumption is necessary.

Quantifier order and convergence mode are part of the claim. For example, pointwise convergence in probability over a declared class $\mathcal{P}$ has the pattern

$$\forall P \in \mathcal{P} \forall \varepsilon, \eta > 0 \exists N(P, \varepsilon, \eta) \forall n \geq N(P, \varepsilon, \eta) : \Pr_P\{|\hat{F}_n - F(P)| > \varepsilon\} < \eta. \quad (2)$$

A uniform-in- P statement would require an $N(\varepsilon, \eta)$ that works for every law in $\mathcal{P}$. Equation (2) does not imply that stronger statement. Likewise, $\forall \varepsilon \exists N$ cannot be replaced with $\exists N \forall \varepsilon$. The claim packet records this order before proof search so that a correct proof of a weaker rearrangement is not mistaken for completion.

An obligation structure is an AND/OR directed acyclic hypergraph. Its nodes are propositions or finite checks. An AND-hyperedge $\{u_1, \dots, u_r\} \rightarrow v$ records that all named premises are needed to derive v ; alternative incoming hyperedges are OR-routes, any one of which may derive v . A leaf closes only through accepted evidence of the declared class. An internal node closes only when every tail node of at least one accepted incoming hyperedge is closed and the edge's implication has itself been justified. Define each route as a set of dependency nodes in a frozen admissible vertex universe. Unless a deliberately trivial cut is reported separately, that universe excludes the common claim/goal node and synthetic route or AND/OR aggregator nodes. The packet retains every minimal shared cut set: each inclusion-minimal admissible node set that intersects every route in a declared route family. The packet says whether that family is candidate, permitted, or accepted and tags every cut node as open, closed, or a known common cause. An accepted route cannot contain an open node; a cut over accepted routes may nevertheless expose a closed shared dependency. Rephrasing an open node, or closing one branch of an AND-edge, is not closure.

OBLIGATION GRAPH

Accepted routes, AND prerequisites, and all minimal cuts

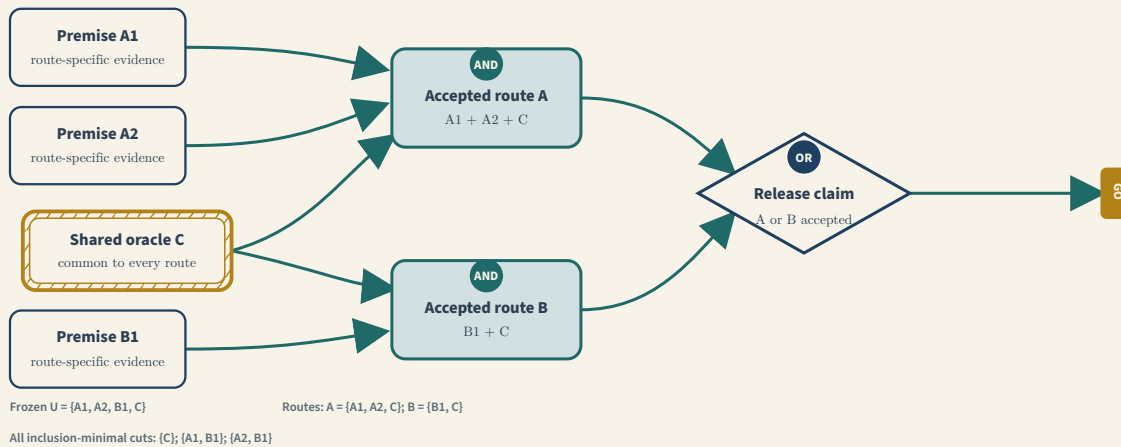


Figure 2: AND prerequisites, OR routes, and the complete three-cut family in the frozen example.





1.4 Certificates and independently checkable computation

Exploratory floating-point output can suggest a lemma but is not a proof. A certificate separates generation from checking:

$$\text{instance} \xrightarrow{\text{generator}} \text{certificate} \xrightarrow{\text{separately specified checker}} \{\text{accept, reject}\}.$$

The checker must state the finite proposition that acceptance establishes. For rational inputs, integer and rational identities should remain exact. Transcendental terms require either a symbolic identity or outward-rounded intervals. If $L \leq x \leq U$ and a sign is needed, then

$$L > 0 \implies x > 0, \quad U < 0 \implies x < 0, \quad 0 \in [L, U] \implies \text{unresolved}.$$

The last branch is not a failure; it is the correct abstention. A useful mutation test changes a sign, endpoint, mask, or denominator in a copy of the evidence. The checker must reject the mutant for the intended reason.

1.5 PID vocabulary and a finite worked reconstruction

PID is a family of proposals for decomposing how several sources $S_1, \dots, S_m$ provide information about a target T . This worked reconstruction concerns a finite redundancy-lattice PID after its cumulative functional and order have been declared; it does not assert that every PID proposal uses this carrier or semantics. For finite variables, a joint law P assigns nonnegative masses summing to one. A source collection is a nonempty subset of $\{1, \dots, m\}$. An antichain is a nonempty set of source collections in which no member contains another. The declared cumulative quantities c_α , indexed by antichains, are related to atoms π_β by a finite zeta transform

$$c_\alpha = \sum_{\beta \preceq \alpha} \pi_\beta, \quad \pi_\alpha = \sum_{\beta \preceq \alpha} \mu(\beta, \alpha) c_\beta, \quad (3)$$

where $\preceq$ is the declared redundancy order and μ is its Möbius function. Equation (3) is finite algebra. It does not by itself prove that the cumulative values have the intended paper semantics or that floating-point code evaluates them accurately.

For the ordinary two-source four-atom shape, let R , U_1 , U_2 , and S denote redundancy, source-specific unique terms, and synergy. If the corresponding cumulative values are c_R, c_1, c_2, c_{12} , inversion is

$$\begin{aligned} R &= c_R, \\ U_1 &= c_1 - R, \\ U_2 &= c_2 - R, \\ S &= c_{12} - R - U_1 - U_2. \end{aligned}$$

Substitution gives the reconstruction identities

$$c_1 = R + U_1, \quad c_2 = R + U_2, \quad c_{12} = R + U_1 + U_2 + S. \quad (4)$$

An exact checker can verify (4) for every declared finite count table. That closes the finite reconstruction obligation only and proves no sign theorem. For the categorical Makkeh–Gutknecht–Wibral shared-exclusions functional specifically, the exact-real informative and misinformative component atoms are separately nonnegative at the defined positive-mass pointwise realizations, and their finite-PMF averages inherit nonnegativity, while the corresponding signed nets may be negative (Makkeh et al., Theorem IV.3). pid-rs therefore does not clamp those signed-net atoms. Williams–Beer $I_{\min}$ instead has its own exact-real finite-PMF nonnegativity





theorem for its atoms; a materially negative value from that pid-rs plug-in route is an implementation defect, not an MGW-style signed-net interpretation. Binary64 roundoff remains a separate bounded numerical obligation in both cases. Neither sign result is a property of Möbius inversion itself or transfers to Ehrlich continuous shared-exclusions/PID2 estimates, KSG mutual-information estimates, project heuristics, or an arbitrary PID functional.

1.6 Negative example: an unseen rare cell

Let a finite population contain a cell z_* with mass $P(z_*) = \rho > 0$. Under i.i.d. sampling from P , the exact probability that z_* is absent from n rows is

$$\Pr\{\hat{P}_n(z_*) = 0\} = (1 - \rho)^n. \quad (5)$$

With $\rho = 10^{-4}$ and $n = 1000$, this probability is approximately 0.9048. Thus an observed support can omit a genuine population cell with high probability. Replacing an unknown population mass floor with the smallest observed mass is therefore invalid without another assumption or confidence construction. Fixed-support continuity remains useful, but it is not a support-discovery theorem or a practical finite-sample certificate.

1.7 Negative example: pairwise is not mutual independence

Let X and Y be independent fair bits and set $Z = X \text{ xor } Y$. Every pair among X, Y, Z is independent, but the triple is not: Z is determined by X, Y . Therefore pairwise independence, zero covariance, or a benign autocorrelation plot cannot establish mutual independence of complete rows within a dependence color. A concentration result whose premise is mutual within-color independence needs that premise justified by design or a separate theorem.

1.8 Bounded evolutionary search: useful scope and hard boundary

Genetic or evolutionary search can be useful when fitness is mechanically checkable. For a mutation m of a proof script, certificate, or implementation, a search may rank

$$f(m) = (\mathbf{1}\{\text{invalid mutant is accepted}\}, -\text{witness size}, -\text{runtime})$$

lexicographically. Mutation operators may swap a source mask, replace a union with an intersection, reverse a log ratio, perturb an interval endpoint, or delete one Möbius predecessor. Any accepted invalid mutant is a concrete checker weakness and must be retained and minimized.

Failure to find such a mutant is not a proof. An exhaustive finite mutation domain $\mathcal{M}_B$ can support only the bounded statement

$$\forall m \in \mathcal{M}_B, \quad \text{Checker}(m) = \text{reject}.$$

A heuristic genetic run samples only part of its search space. Its valid output is a discovered counterexample or a diagnostic search log, never a theorem-vote or a confidence score.





2 Typed assurance and rigorous evidence aggregation

2.1 Status of this supplement

This is a typeset-only explanatory supplement to the canonical protocol in Section 3. It specifies how evidence records may be aggregated without turning heterogeneous proof, numerical, software, statistical, and consumer evidence into an unsupported score. It is process guidance, not a calibrated confidence model for `pid-rs`.

2.2 Preserve a vector across assurance layers

Evidence has different semantics at different layers. Preserve a typed vector

$$\mathcal{A}(H) = (E_{\text{sem}}, E_{\text{math}}, E_{\text{formal}}, E_{\text{num}}, E_{\text{exec}}, E_{\text{stat}}, E_{\text{consumer}}, E_{\text{archive}}) \quad (6)$$

for a fully written proposition H . Each component is itself a record

$$E_\ell = (\text{state}, \text{scope}, \text{assumptions}, \text{artifacts}, \text{negative evidence}, \text{residual boundary}). \quad (7)$$

The component *artifact-verification state* is one of **not-applicable**, **open**, **rejected**, or **closed**. It is distinct from the accepted evidence class (for example, theorem, finite machine check, certified bound, empirical result, counterexample, or diagnostic) and from the claim disposition (**proposed**, **active**, **blocked**, **falsified**, or **complete**). Multiple evidence records may support one component; one claim has one current disposition.

The vector and gate map is:

Component	Gate(s)	Required subject
E_{sem}	G0–G1	claim identity, conventions, and premises
E_{math}	G2	mathematical proof or counterexample
E_{formal}	G3	proof-checker semantics and implication
E_{num}	G4	rigorous numerical enclosure
E_{exec}	G5	executable conformance
E_{stat}	G6	scoped estimator calibration
E_{consumer}	G7	consumer contract and qualification
E_{archive}	G8	reproducible evidence and release archive

A machine-checked lattice identity and an open calibration study are not numbers on one common scale. Do not map their labels to integers and average them. An average would erase the fact that one load-bearing layer is open. The decision rule is instead proposition- and layer-specific: every applicable required component must meet its stated closure condition, and every inapplicable component needs a recorded reason.





2.3 Critical cut sets determine dependence

Let test families $T_1, \dots, T_k$ bear on a binary proposition H , and let C range over a fully defined finite set of latent shared-failure states for an oracle, imported theorem, generator, or implementation. Different test names do not make them independent. For each $B \in \{H, \neg H\}$, a complete joint evidence model must retain the shared state:

$$\Pr(T_1, \dots, T_k \mid B) = \sum_c \Pr(C = c \mid B) \Pr(T_1, \dots, T_k \mid B, C = c). \quad (8)$$

Multiplying marginal pass probabilities is justified only after an evidence-dependence model establishes the required conditional independence under both H and $\neg H$. A critical-cut-set memo names each shared node, which routes use it, and which dependency-disjoint evidence—if any—closes it. If the latent failure-state space omits an identified common cause, the numerical model is incomplete and the scalar route must abstain.

2.4 Worked example: five correlated test families are one confirmation at a shared oracle

Suppose five suites exercise row permutations, source permutations, small count tables, canonical logic gates, and randomized tables. All five compare the Rust result with the same reference oracle. Let D mean that the implementation has a particular semantic defect, and let O mean that the shared oracle also contains the same defect. Conditional on $D \wedge O$, every suite can pass:

$$\Pr(T_1 = \dots = T_5 = \text{pass} \mid D, O) = 1. \quad (9)$$

They are useful for code paths and transformations outside the shared defect, but at the oracle cut set they are one correlated route.

For scale only, consider a hypothetical, fully specified model in which $\Pr(O \mid D) = q = 0.01$, while in the absence of O each suite independently misses D with probability $\varepsilon = 0.01$. Then

$$\Pr(\text{all five pass} \mid D) = q + (1 - q)\varepsilon^5 \approx 0.010000000099, \quad (10)$$

not $\varepsilon^5 = 10^{-10}$. Ignoring the shared oracle would overstate the strength against this defect by about 10^8 . The numbers in (10) are an illustration, not calibrated **pid-rs** probabilities. If q and the conditional miss rates have not been externally calibrated, no numerical confidence follows.

The retained raw evidence should say, for example:

Evidence component	Established locally	Shared boundary retained
E_{exec}	Five transformation and corpus families pass	All use oracle C
E_{num}	Oracle values reproduce their own fixtures	No separately derived enclosure yet
E_{formal}	Finite reconstruction lemma closes	Does not identify oracle semantics
Candidate separate route	Open until a separately written exact or interval checker passes and its dependency vector is audited	Must not import C 's result table

This vector remains in the evidence packet even if a permitted scalar is later reported.



EVIDENCE INDEPENDENCE

Five routes may still be one route semantically

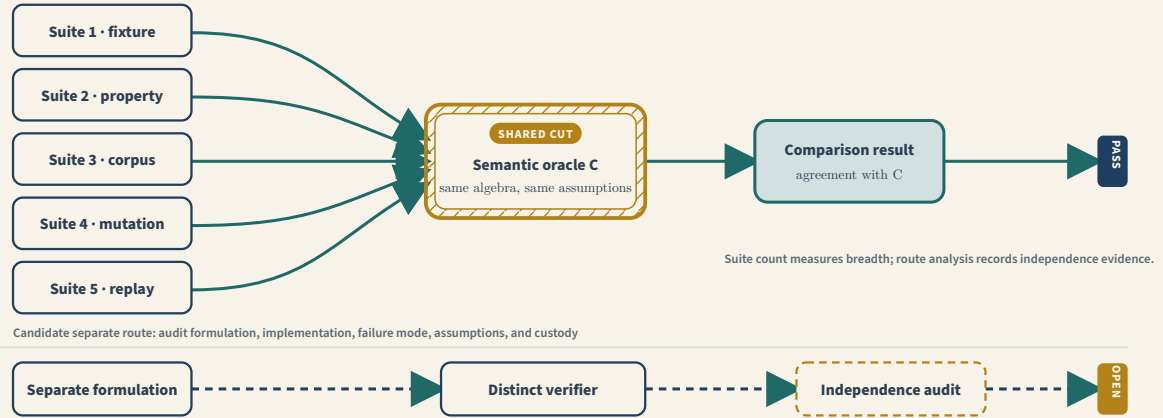


Figure 3: Five suite names do not imply five independent semantic routes when every suite uses one oracle.

2.5 When a model-conditional posterior belief is permitted

A scalar is prohibited for a vague project, method, or release. It is permitted only for one fully specified empirical decision proposition H with a truth-labelled external reference class. It is not an operational replacement for deductive truth of a fixed theorem. The packet must include all of the following:

1. an explicit prior $\pi = \Pr(H)$, with provenance and interpretation;
2. calibrated likelihoods $L_\theta(E | H)$ and $L_\theta(E | \neg H)$, defined under one declared evidence probability-mass or density convention and, for densities, one declared dominating measure;
3. an evidence-dependence model, including every identified shared cut-set node;
4. truth-labelled external calibration data not reused to construct, select, stop, or tune the evaluated evidence;
5. a declared reference class, exchangeability or transport assumption, domain-shift model, and the complete frozen evidence-selection and stopping protocol;
6. finite-calibration uncertainty and model-shift uncertainty inside a nonempty sensitivity region Θ , together with uncertain priors, likelihoods, and dependencies;
7. preregistered decision thresholds and an abstention rule; and
8. the complete raw assurance vector (6)–(7) beside the scalar.

For every $\theta \in \Theta$, require $0 \leq \pi_\theta \leq 1$, finite nonnegative measurable likelihood values under that common convention, and

$$0 < d_\theta := \pi_\theta L_\theta(E | H) + (1 - \pi_\theta) L_\theta(E | \neg H) < \infty.$$

If any of those premises fails, the scalar route abstains. For a fixed admissible θ , Bayes' rule gives

$$p_\theta = \Pr_\theta(H | E) = \frac{\pi_\theta L_\theta(E | H)}{\pi_\theta L_\theta(E | H) + (1 - \pi_\theta) L_\theta(E | \neg H)}. \quad (11)$$



Because Θ is nonempty and every p_θ is well-defined in $[0, 1]$, report the possibly nonattained sensitivity bounds, not a selected point:

$$[p_-, p_+] = \left[\inf_{\theta \in \Theta} p_\theta, \sup_{\theta \in \Theta} p_\theta \right]. \quad (12)$$

If this interval crosses a preregistered decision boundary, or if any required likelihood or dependence term lacks calibration, the result is **abstain**. No arithmetic on ordinal labels may be substituted for (11)–(12). Report p_θ as a *model-conditional posterior belief*, not generic correctness confidence. Even when a scalar is admissible, it never closes a deductive or formal obligation, erases a blocked layer, or enlarges the proposition's scope.

2.6 Change invalidation and publication state

Orient every dependency-bearing edge from prerequisite to dependent. If $(u \rightarrow v)$, the current validity of (v) relies on (u) ; changing (u) invalidates (v) and every other dependent reachable from (u) . It does not invalidate unreachable upstream artifacts. This orientation makes the affected set an ordinary forward reachability query and prevents the common mistake of walking the graph backward.

A publication claim begins **open**, becomes a local **candidate** only when its exact subject tree and applicable gates are bound, becomes **hosted** only when a hosted run supplies a receipt for an identical or descendant commit, and becomes **published** only when that receipt also binds the relevant inputs, workflow identity, and output hashes. Descendancy alone is not sufficient: the receipt must show that the subject artifacts were unchanged or must bind the new exact subject. A later prerequisite change returns every reachable claim to **open/stale**; publication history remains archived, but the old claim is no longer current.

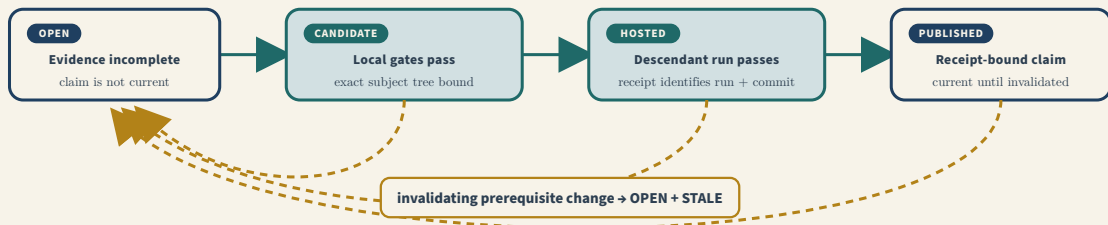
CHANGE CONTROL

Prerequisites point toward dependents; invalidation follows the arrows

EDGE ORIENTATION: prerequisite $\rightarrow$ dependent



PUBLICATION STATE: forward only with bound evidence; reopen when a prerequisite changes



Nodes unreachable from this change are not invalidated by it; their prior state remains separately governed.

Figure 4: Correct edge orientation makes invalidation flow from changed prerequisites to reachable dependents; publication is a receipt-bound state, not a permanent property.





3 Mathematical problem-solving workflow for pid-rs

3.1 Status and scope

The canonical note has two parts. The first records observations from external sources. The second defines a protocol for pid-rs. The typeset LaTeX/PDF companion adds a reader's primer and an evidence-aggregation supplement before reproducing this canonical text byte-for-byte. Those supplements explain the protocol but do not change it. A source observation is not a pid-rs result.

This note does not validate the mathematical claims in the external sources. It also does not claim that the source workflow guarantees a correct proof. The workflow is useful because it makes the claim, search, failure, and audit steps explicit.

This note is project-defined process guidance. It has no software implementation or defining method paper. It adds no PID method, estimator, theorem, benchmark result, or validation.

The protocol is deliberately standalone for a Codex-like agent that resumes after context compaction. A continuity record preserves work state, but it is never authority for a theorem, repository fact, process state, or external fact: the resuming agent verifies and reconciles those facts read-only against their live sources before acting. The durable-agent-continuity section below is mandatory for consequential multi-step work.

The central rule is:

An AI model can propose, refine, or attack an obligation. Only retained and replayable evidence can close the obligation.

Model confidence, polished prose, and agreement among models are not evidence classes.

3.2 Compact glossary

- **PID (partial information decomposition):** a family of decompositions of information that several sources provide about a target. In the ordinary two-source redundancy-lattice grouping, the four terms are redundancy, two source-specific unique terms, and synergy; multivariate redundancy-lattice PIDs generally have finer antichain-indexed atoms. The name does not select a redundancy functional or transfer axioms among PID proposals.
- **Categorical SxPID:** the Makkeh–Gutknecht–Wibral finite-discrete, pointwise shared-exclusions construction and its average under a finite PMF. The population functional accepts a declared finite population PMF. pid-rs' direct row-data path forms the empirical PMF $\hat{p}_n$ and returns $F(\hat{p}_n)$ for two through four sources. If the empirical law itself is the scientific target, this is a descriptive empirical functional; if $F(p)$ for an underlying population law is the target, it is a plug-in estimator. The API does not choose between those targets. Binary64 representation error remains a separate numerical question. Here “categorical” means finite category-valued random variables, not category theory.
- **General measure-theoretic shared exclusions:** Schick-Poland et al.'s proposed auxiliary-indicator/regular conditional probability (RCP)/Radon–Nikodym construction for a finite source family under the paper's locally compact Radon/Borel, metrizable-compact-subset, and complete-measure premises. Under standard conditioning, $P(R) > 0$ gives $P(T \in A \mid R) = P(T \in A, R)/P(R)$. The reviewed arXiv v2 finite-discrete recovery display





instead writes the unnormalized numerator in Section 4.3.1 (physical PDF pages 13–14), so exact recovery of MGW requires that missing normalization or a clarified definition; this is a retained source-obligation, not an author-correction claim. Local values may be extended-real, global claims need their integrability premises, a noninjective random variable has no pointwise inverse, eventwise RN derivatives need a simultaneous countably additive kernel/version theorem, a Borel isomorphism does not imply the atom-plus-Lebesgue density decomposition displayed with Corollary 3.1 (physical PDF page 9; a Cantor law is a singular-continuous diagnostic), and target-local RN values need representative control even on Euclidean spaces. Evaluation at a null indicator value is not version-invariant from almost-everywhere RCP uniqueness alone, and the reviewed bicontinuity step does not prove arbitrary-measurable-bijection invariance (the reviewed bicontinuity step is in Section 4.3.3, physical PDF page 16). pid-rs implements neither the general route nor a practical general estimator.

- **Continuous shared exclusions:** Ehrlich et al.’s main practical, gauge-dependent continuous density/quasi-density formula and source-disjunction kNN estimator are for purely continuous variables. Appendix B also gives mixed logical-statement quasi-density examples, and Appendix K sketches a mixed estimator ansatz plus one symbolic example while stating that mixed systems exceed the developed estimator’s capabilities; it supplies no demonstrated or calibrated mixed estimator. pid-rs does not implement a general mixed route. The disjunction is not a probability union law, and its quasi-density need not integrate to one. The practical route is inspired by, but not identical to, the Schick-Poland construction and is default-off in pid-rs. The paper’s matched refining-bin calculation is a premise-bound asymptotic motivation, not identity of the constructions, a general convergence proof, or a convergence theorem for a fixed pid-rs quantizer. Its reviewed Eq. (8) overlap display (physical PDF page 4) repeats m_{S_2} ; independent substitution of both source-bin widths requires $m_{S_1} m_{S_2}$, a local source correction rather than an estimator change. Definition 1 and the explicit non-normalization discussion are on physical PDF page 5; the mixed-system limitation and Appendix K begin on physical PDF page 27. Retain three further arXiv-v3 source corrections without silently converting them into pid-rs estimator changes. First, the global two-source expectation immediately after Definition 2 on physical PDF page 5 prints $dt ds_1 ds_1$. The density and local integrand depend on (t, s_1, s_2) , and the Appendix D derivation on physical PDF page 24 uses $dt ds_1 ds_2$, so the second displayed differential must be ds_2 . Second, Equation (14) on physical PDF page 10 is a natural-log/digamma expression. To reproduce the paper’s base-2 definitions and bit-valued examples, the complete right-hand side must be divided by $\ln 2$; pid-rs intentionally retains the natural-unit expression and converts an upstream bit fixture by $\text{nats} = \text{bits} \cdot \ln 2$. Third, Algorithm 6 on physical PDF page 28 must pass the antichain α to both `_compute_epsilons(S,T,antichain)` and `_compute_n_alpha(S,antichain,eps)`, and target counts must come from `_compute_n_T(T,eps)`. The printed pseudocode omits α twice and calls the source count routine on T . Algorithm 5 also advertises an unused α argument, while Algorithm 6’s result header describes distances although the algorithm returns the redundancy scalar. The authors’ pinned code supplies this corrected executable wiring and divides by $\ln 2$ when it reports bits. That code is correlated with the defining-paper route: it helps disambiguate units and intended calls but does not prove estimator consistency, calibration, population-support validity, or refinement by pid-rs.
- **Colored PID:** not a separate functional. A dependence coloring qualifies a sampling theorem, not PID.
- $I_{\min}$: the Williams–Beer minimum-specific-information redundancy functional.
- **KSG:** the Kraskov–Stögbauer–Grassberger nearest-neighbour mutual-information esti-





mator. Their higher-dimensional use of “redundancy” denotes multi-information/total correlation, not a PID redundancy functional or atom. For a scientific population-estimation claim, pid-rs uses a conservative supported route requiring i.i.d., unrounded rows from one fixed law; full-dimensional absolute continuity of every required marginal and joint law; finite MI; and explicit boundary, density/smoothness, ambient-coordinate metric, and local-geometry premises. This is the local fail-closed contract, not a claim that no different KSG theorem could weaken a premise. Observed uniqueness and a support declaration do not prove those premises.

- **Estimand:** the exact population quantity a procedure is intended to estimate.
- **Oracle:** a reference implementation or value source used to adjudicate another route. An oracle is evidence only within its stated construction and error contract.
- **Antichain and redundancy order:** an antichain is a nonempty collection of nonempty source subsets in which no member contains another. The explicitly typed finite partial order indexes PID cumulatives. This invokes neither category theory nor an infinite-poset inversion theorem.
- **Zeta and Möbius transforms:** inverse finite linear transforms on the declared finite antichain poset between lattice cumulatives and atoms. A matrix identity alone does not identify the intended event semantics. Infinite-poset use would require a separate theorem with local finiteness or convergence premises.
- **Dependence coloring:** a partition of sample rows into color classes such that the complete rows within each class satisfy the theorem’s declared mutual-independence premise.
- **Outward-rounded interval:** endpoints rounded down and up so the exact real value is enclosed.
- **binary64:** the IEEE 754 double-precision floating-point format.
- **MPFR and Arb:** libraries for specified-rounding multiprecision arithmetic and rigorous ball arithmetic, respectively.
- **Lean and SMT:** Lean is a proof assistant with a small proof-checking kernel; satisfiability modulo theories (SMT) solvers decide or search within supported logical theories.
- **Complete separation oracle:** a total algorithm on an exactly declared domain, with a proved termination/resource bound, that returns either a violating object or a checkable certificate that no violator exists. A search guaranteed only to find a violator if it eventually returns is a falsifier search, not a universal-closure oracle.
- **Confidence limits and multiplicity control:** sampling-uncertainty bounds and procedures that control an error criterion across multiple reported hypotheses or cells.
- **Controlled versus blind holdout:** a controlled holdout freezes and separates development from evaluation; it is blind only when the named development roles cannot access sealed rows, seeds, targets, keys, or unredacted results before the first complete adjudication.
- **Digest:** unless a claim says otherwise, a digest is lowercase SHA-256 over the exact raw bytes of the named artifact. Semantically equivalent re-encodings have different digests.
- **Ecosystem projects:** Prisoma analyzes learned representations; Galadriel monitors cross-sensor consistency; Haldir concerns authorization/reference monitoring; and Crebain is a sensor-fusion visualization consumer. Their versioned requirements are tracked in `ECOSYSTEM_CAPABILITIES.md`.





3.3 Primary-source pins for PID vocabulary and routing

The source-specific observations and routing boundaries in this note use the exact arXiv revisions below. On 2026-08-04, each linked arXiv submission history ended at the listed revision. That is a dated discovery observation, not immutable byte custody: the source PDFs are not repository artifacts, and a provider can regenerate delivered PDF bytes without changing an arXiv version label. “Reviewed” therefore identifies the cited revision, not a claim that an arXiv PDF and a linked journal edition are byte-identical.

Local family or use	Exact primary source reviewed and separately linked publication record
MGW categorical shared exclusions	Makkeh, Gutknecht & Wibral, “Introducing a differentiable measure of pointwise shared information,” arXiv:2002.03356v5, revised 30 Mar 2021; separately, <i>Physical Review E</i> 103, 032149 (2021)
Ehrlich continuous shared exclusions	Ehrlich et al., “Partial Information Decomposition for Continuous Variables based on Shared Exclusions: Analytical Formulation and Estimation,” arXiv:2311.06373v3, revised 27 Mar 2024; separately, <i>Physical Review E</i> 110, 014115 (2024)
Schick–Poland general measure-theoretic construction	Schick–Poland et al., “A partial information decomposition for discrete and continuous variables,” arXiv:2106.12393v2, revised 24 Jun 2021
Williams–Beer $I_{\min}$	Williams & Beer, “Nonnegative Decomposition of Multivariate Information,” arXiv:1004.2515v1, submitted 14 Apr 2010
KSG mutual-information estimator	Kraskov, Stögbauer & Grassberger, “Estimating Mutual Information,” arXiv:cond-mat/0305641v1, submitted 28 May 2003; separately, <i>Physical Review E</i> 69, 066138 (2004) and its 2011 erratum to Appendix Eq. (A5). The erratum says that error does not affect the paper’s other results; pid-rs does not use the corrected appendix extremum claim as an estimator-validity bridge.
Shannon-invariant screening quantities	Gutknecht et al., “Shannon invariants: A scalable approach to information decomposition,” arXiv:2504.15779v1, submitted 22 Apr 2025
Barà mixed discrete-target/continuous-source route	Barà et al., “Partial information decomposition for mixed discrete and continuous random variables,” arXiv:2409.13506v1, submitted 20 Sep 2024
Lyu–Clark–Raviv published theorem package	“Multivariate Partial Information Decomposition: Constructions, Inconsistencies, and Alternative Measures,” arXiv:2508.05530v2, revised 11 Feb 2026; separately, <i>Physical Review E</i> 113, 034102 (2026)
Lyu–Clark–Raviv structural preprint	“Structural Impossibility of Antichain-Lattice Partial Information Decomposition,” arXiv:2604.03869v2, revised 14 Apr 2026; no journal-edition claim is made here

3.4 Source observations

3.4.1 X thread

The thread author reports six solutions to open Erdős problems in five days. The thread does not, by itself, verify those solutions. It describes this repeated loop:





```
attempt -> failure -> diagnosis -> new approach -> proof draft -> adversarial audit ->
↔ revision
```

The author reports runs of about 6 to 32 hours for individual problems. The author also gives these search rules:

- Restate the exact problem.
- State what a complete proof or disproof must show.
- List weaker results that do not solve the problem.
- List traps and edge cases.
- Start several independent approaches.
- Keep incompatible approaches active.
- Search for counterexamples to proposed lemmas.
- Block a route that stops at an unproved statement of comparable strength.
- Challenge each candidate argument before acceptance.

These observations come from the five linked posts:

- [Thread introduction](#)
- [Exact problem and audit requirements](#)
- [Independent routes and blocker rules](#)
- [Long-run work pattern](#)
- [Research loop](#)

3.4.2 Cycle Double Cover prompt

The [Cycle Double Cover prompt](#) starts with exact definitions. It fixes the graph class, the meaning of a cycle, edge multiplicity, disconnected graphs, and the empty graph. It then states the one result that counts as complete.

The prompt also lists results that do not count. Examples include proofs for special graph classes, finite computation, a cover with the wrong edge multiplicity, and a reduction to another unproved claim. It asks for independent approach families. It delays the sharing of ideas between routes until each route has clear strengths and gaps. It requires concrete lemmas, constructions, equations, or counterexamples. It also gives a problem-specific audit list.

Project interpretation: this structure separates three questions:

1. What is the exact claim?
2. What work can help the search but cannot complete the claim?
3. What checks can falsify a candidate proof?





3.4.3 Erdős repository

The source tree was inspected at immutable commit [f2ae0edb45cbdb257e135d51ef855f64caeb348b](#). At that commit, each of the six problem directories contains a problem-specific prompt and a paper. The tree also contains Lean projects for problems 486, 788, and 1038. It contains Python numerical verifiers for problems 390, 536, and 1038.

The immutable prompt files are available for [1002](#), [1038](#), [390](#), [486](#), [536](#), and [788](#). The prompts state the quantifier order and required target strength explicitly. They list non-solutions and problem-specific failure modes. Examples include:

- an exact asymptotic limit instead of an order bound;
- one fixed infinite witness instead of a different finite witness at each scale;
- a full-sequence limit instead of a selected subsequence;
- exact extrema and sharpness instead of numerical candidates;
- strict control of limit interchange, tightness, and mass escape;
- preservation of integer, graph, or measure structure during a reduction.

Project interpretation: problem 1038 shows one certificate pattern. Its [numerical verifier](#) is designed to use a double-precision scan for initial brackets. Its higher-precision and Arb interval checks are designed to certify signs. The [Lean project](#) contains rational and interval data for finite kernel checks. This review did not run the verifier or build the project. The repository also has immutable [486](#) and [788](#) Lean trees. It has numerical verifiers for [390](#) and [536](#).

Project interpretation: no complete chronological failure log was found in the inspected commit. The prompts request a route registry and blocker policy, but this inspection did not find a full transcript. This difference matters when pid-rs records its own process.

3.4.4 Erdős 477 cubic repository

The [pw/erdos477-cubic](#) repository was inspected at immutable commit [8440d599890b5a5ef7b212c65338723ab2443eaf](#). This was a process review only. It did not validate the repository's claimed theorem.

Five process choices are useful for pid-rs:

1. The repository pins a semantic ambiguity against the primary problem statement before using the proof. In its case, it distinguishes uniqueness of an image value from uniqueness of a parameter that enumerates that value.
2. It preserves the earlier, narrower cubic argument when the claim expands to all higher powers. The narrower route remains a worked instance and an independent audit target instead of being overwritten by the generalization.
3. Its [VERIFICATION.md](#) separates elementary bookkeeping from two imported determinant-method results. It explicitly says that two proof narratives that share the same imported crux are correlated evidence, not two independent confirmations.
4. It retains the actual adversarial briefs for the [cubic claim](#) and the [generalized claim](#). Each brief names concrete failure targets, asks for refutation rather than confirmation, and distinguishes a surviving argument from an independently re-derived one.





5. An adversarial pass found an incorrect genus formula. A cheap invariant exposed it: the old formula produced a non-integer genus in one case. The repository corrected the formula, identified the exact dependency subgraph that used it, and recorded why the main theorem did not depend on that subgraph.

The same record also shows limits that pid-rs should not copy. The construction and adversarial passes used the same model family. The raw sessions are not retained in the repository. No formal proof or human specialist review is present. Its primary-source checks establish that cited statements exist and that an application appears to meet their hypotheses; they do not re-prove the cited determinant methods. Therefore, labels such as “full proof” in that repository are not evidence for pid-rs and do not change any pid-rs disposition.

Project interpretation: four transferable controls should be added below - critical-cut-set accounting for correlated routes, an imported-theorem application map, a load-bearing correction ledger, and cheap invariant probes. These controls improve auditability without importing the external mathematics.

3.4.5 Corrected vector-bundle claim: a typed citation-edge failure

The public correction chain beginning with the [original claim](#), the [linked draft](#), Tony Feng’s [specific objection](#), the author’s [acknowledgement](#), and the conspicuous [correction post](#) was inspected on 2026-07-25. The downloaded 18-page draft had SHA-256 `ebb5aa2c8d1d08cd1c7692ac0526cf537c00985bf5e129ff165be128404e69ca`. The three distinct images visible in the claim/correction chain were inspected at their original resolutions: a paper-introduction image, a screenshot of Theorems A and B, and a screenshot of the correction. They repeated claims or text present in the draft/thread and supplied no independent proof.

The dedicated Chrome-extension automation transport failed twice. Codex Computer Use subsequently loaded the signed-in public thread in Chrome and exposed its current visible conversation/replies tree. Public status responses, the first public replies page, the linked draft, and the primary cited paper were used as corroborating retrieval paths; reproducing the same content does not make them independent mathematical routes. The public replies page was a third-party transport and returned 20 items, including nested responses; its pagination cursor did not advance. These access details are session observations, not a retained browser attestation. This record therefore covers the mathematical exchange and follow-ups visible through those paths, but does not claim completeness for deleted, private, later, or unreturned replies. Social agreement, criticism, or silence was not treated as mathematical evidence.

The load-bearing error is narrower and more informative than “an AI proof was wrong.” The draft’s Theorem 7.1 uses the exact sequence

$$\pi_{10,5}^{\mathbb{A}^1}(S^{9,5})(\mathbb{C}) \longrightarrow \pi_{9,5}^{\mathbb{A}^1}(SL_4)(\mathbb{C}) \longrightarrow \pi_{9,5}^{\mathbb{A}^1}(SL_5)(\mathbb{C}).$$

It then reads Theorem 7.2.1 of the immutable arXiv v3 source [2306.04631v3](#), downloaded here with SHA-256 `d4bd95572c7e2c356e964407ceab26c64c37768a655b45b45feaa4ab50dd8536`, whose displayed short exact sequence is

$$0 \longrightarrow K_{d+2-j}^M/24 \xrightarrow{(\nu_d)_*} \pi_{d+j,j}^{\mathbb{A}^1}(S^{2d-1,d}) \longrightarrow GW_{d+1-j}^{d-j},$$

followed by the grammatically ambiguous words “which is an isomorphism if $j \geq d - 3$.” The draft attached “is an isomorphism” to the left map $(\nu_d)_*$ and, at $d = j = 5$, asserted





$$\pi_{10,5}^{\mathbb{A}^1}(S^{9,5})(\mathbb{C}) \cong K_2^M(\mathbb{C})/24 = 0.$$

The preceding source discussion, together with Theorem 7.2.2(3), assigns the range-dependent surjectivity property to the rightmost map to the Grothendieck–Witt sheaf, not an isomorphism property to $(\nu_d)_*$. Theorem 7.2.1’s word “isomorphism” remains grammatically and mathematically defective in the displayed formulation, so this review does not repair it by choosing whichever referent makes the draft’s argument work.

There is a stronger source-based check. The proof of Theorem 7.2.1 invokes the stable 1-line calculation of Røndigs–Spitzweck–Østvær. Its immutable [1604.00365v2](#) artifact, downloaded here with SHA-256 `bdcf6f0ef128457740c09c5fb38c1a187951b29119d5ec2b8ba0339cb7887966`, states in equation (1.1) and Theorem 5.5, for the relevant fields and range, the exact sequence

$$0 \longrightarrow K_2^M(F)/24 \longrightarrow \pi_{1,0} \mathbf{1}(F) \longrightarrow F^\times / (F^\times)^2 \oplus \mathbb{Z}/2 \longrightarrow 0.$$

ABH’s stable-range comparison specializes the middle sheaf evaluation at $d = j = 5$ to this stable group. Over $F = \mathbb{C}$, both outer arithmetic simplifications are exact: every Milnor symbol $\{a, b\}$ is $24\{a, b\}$ after choosing $\alpha^{24} = a$, so $K_2^M(\mathbb{C})/24 = 0$; and every nonzero complex number has a square root, so $\mathbb{C}^\times / (\mathbb{C}^\times)^2 = 0$. The remaining summand is not zero. Consequently,

$$\pi_{10,5}^{\mathbb{A}^1}(S^{9,5})(\mathbb{C}) \cong \mathbb{Z}/2.$$

Equation (27) is therefore false, rather than merely unsupported. This is an evaluation of motivic homotopy sheaves on $\text{Spec}(\mathbb{C})$; it is not the separate Betti or complex- realization argument later in the draft. The characteristic and field hypotheses of both cited results apply to $\mathbb{C}$.

The specialization itself supplies the smallest retained human-readable counterexample to the wrong adjacent-arrow inference:

```
0 -> 0 -> Z/2 --id--> Z/2 -> 0
```

The right nonzero map is an isomorphism, while its adjacent map from zero is not and the middle group is nonzero. The executable checker represents the same finite group as $C_2 = \mathbb{Z}/2$,

```
0 -> 0 -> C2 --id--> C2 -> 0
```

and exhausts every element, image, and kernel. The prose and executable witnesses are two representations of one countermodel, not independent evidence routes.

Dependency reach must remain scoped. Equation (27) was used to make the second arrow in (26) injective; after separately arguing that this arrow has zero image, the draft concluded its domain was zero. The corrected first group makes that inference unavailable. Conditional on retaining the draft’s separate zero-image argument, exactness now gives only a surjection

$$\mathbb{Z}/2 \twoheadrightarrow \pi_{9,5}^{\mathbb{A}^1}(SL_4)(\mathbb{C}),$$

so the target group is constrained to be either 0 or $\mathbb{Z}/2$; this audit does not choose between them. The displayed proof of Theorem 7.1 therefore fails, but this calculation does not disprove Theorem 7.1. In the draft’s displayed dependency chain, Theorem 7.1 feeds Corollary 8.3, Theorem 8.4, Theorem A’s no-motivic-lift and nonalgebraizability route, and Corollary 1.1. Earlier $\mathbb{P}^4$ /projective machinery lies outside this particular cut: that means this error does not by itself invalidate it, not that this inspection audited or salvaged it. This inspection did not prove





or disprove non-algebraizability, did not prove that a motivic lift exists, and did not adjudicate the author's later claim that other machinery in the draft may survive.

Seven lenses isolate the transferable process result:

1. **Semantic:** the anaphor “which” had two type-correct-looking referents.
2. **Logical:** a property of one arrow in an exact sequence was transferred to its neighbor.
3. **Source:** checking that a cited theorem exists did not check which morphism its qualifier names.
4. **Retrieval:** a model that was suspicious before lookup reportedly accepted the step after reading the ambiguous source, so retrieval can reinforce rather than remove an error.
5. **Independence:** repeated adversarial readings by the same model family against the same prose shared the same citation edge and count as correlated evidence.
6. **Formal:** a proof assistant can check the wrong imported premise if the human correspondence layer binds the wrong arrow; the seam needs an exact typed signature, not merely more code.
7. **Sociotechnical and scope:** public expert review happened quickly but stochastically. Its success here does not make social-media silence evidence, and one broken route does not by itself invalidate every independent result in the draft.

Project interpretation: add the typed citation-edge gate below. This is PID-neutral process evidence. No vector-bundle theorem, motivic calculation, or alternative PID definition is imported into pid-rs.

3.4.6 External AI discussions

The supplied workflow reports that three linked AI discussions were retrievable on 2026-07-24. They are process examples, not evidence that their mathematical conclusions are correct.

The [Jacobian discussion](#) shows why a review must freeze signs, orderings, normalizations, and exceptional cases before it interprets a calculation. A checked local determinant does not establish a global statement. Likewise, a fixed-support continuity result does not establish a finite-sample rate or a binary64 implementation theorem.

The [unsplittable-flow discussion](#) records a false candidate counterexample. The candidate failed when the full path closure was included. For PID, the corresponding rule is to include all antichains, event unions, supported realizations, branches, permutations, and implementation paths named by the claim.

The [pid-rs correctness discussion](#) is an external review intake. Its recommendations are work items until repository evidence replays them. Its pass or fail language does not change a project claim disposition.

3.4.7 OpenAI-hosted proof-process walkthroughs: process controls only

Two long collections obtained from OpenAI CDN URLs were reviewed at exact source bytes:





Source	Exact reviewed artifact	Coverage boundary
How the Ideas Came Together (reasoning-walkthroughs.pdf)	441,468 bytes; SHA-256 13b95999f060c0be2142089cfb8b17b75e9231c3c1f3fa0980445ff1b35f0b3b; 62 PDF pages	62/62 pages text-reviewed and rendered in page order
Ten Advances in Mathematics and Theoretical Computer Science	2,266,371 bytes; SHA-256 64b900d5fae6fe22f2ae1b8e3b712d20055194a6c81cf343a2455e5898ac7dd6; 249 PDF pages	249/249 pages text-reviewed and rendered in page order; physical and printed page numbers were distinguished

No missing, blank, corrupt, clipped, or misordered source page was observed in the reviewed renders. That is a source-coverage and layout observation, not verification of either collection's theorems. The first collection says on PDF page 2 that its narratives were written by an AI model after access to original chains of thought and resulting papers. The narratives are therefore retrospective process accounts, not reproducible private reasoning traces or primary PID sources. The second collection states field-specific results across harmonic analysis, coding, groups, operator algebras, complexity, quantum information, geometry, Ramsey theory, and related areas. This review did not validate those results. Page anchors below are physical one-based PDF pages.

The two collections are also correlated evidence: their topic and result sets overlap, the first is explicitly retrospective, and both were supplied from the same OpenAI CDN source family. Agreement between them does not create institutional independence, prove that a control was followed in pid-rs, or close a mathematical obligation. Only the following premise-explicit process controls transfer:

Transferable control	Page-anchored source observations	pid-rs use and required boundary
Freeze the exact object, conventions, comparator, and completion claim before search	Reasoning pp. 6–10, 29, 33–36, 50, 55; Ten Proofs pp. 3–12, 29–35, 79–99, 114–117	Bind the PID functional, support, source roles, order, units, representation, and optimized target. A precise statement is not evidence that it is true.
Preserve hypotheses through every reduction and representation change	Reasoning pp. 17–24, 39–40, 47–48, 53–54; Ten Proofs pp. 6, 49–58, 84–94, 96–108, 184–218	Record source and destination objects, preserved premises, cost/scale conversion, and map-back. Similar formulas or output equality are not a mapping theorem.
Make central, tail, endpoint, interior, finite, and optimizer-escape regimes exhaustive	Reasoning pp. 8–14, 31, 41–45, 49, 54, 56–58; Ten Proofs pp. 11–25, 41–65, 154–180, 195–206, 219–227	Cover zero cells, support changes, singularity, finite cutoffs, and nonattained optima. A local or compact-range argument cannot close a global claim.
State limit order, uniform constants, finite thresholds, and subsequence-to-full-sequence bridges literally	Reasoning pp. 8–10, 13–14, 41–45, 54, 56–58; Ten Proofs pp. 13–27, 36–48, 59–76, 164–180, 221–235, 242–248	Freeze which parameter is fixed first, prove uniformity where limits move, and close the finite exceptional range. A sampled hierarchy, subsequence, or asymptotic rate is not an all-index theorem.
Separate attainment from a supremum/infimum and prove witness existence at the claimed level	Reasoning pp. 12–14, 28, 30–32, 49, 56; Ten Proofs pp. 29–48, 59–65, 154–167, 219–227	Do not select an optimizer when only an approximating sequence is known; construct a finite witness before taking an accuracy limit.
Freeze whether one witness must satisfy all obligations or different witnesses are permitted	Reasoning pp. 17–19, 39–40, 47–48; Ten Proofs pp. 85–90, 207–212, 238–242	A source-blind verifier, common law, common split, or common certificate must be shared when the quantifiers require simultaneity. Per-cell or per-obligation witnesses do not imply one joint witness.





Transferable control	Page-anchored source observations	pid-rs use and required boundary
Preserve probability weights, conditioning objects, and representation levels	Reasoning pp. 20–24, 33–36; Ten Proofs pp. 53–58, 96–108, 154–180	Keep empirical-count weights, selection/conditioning mass, source/target roles, scalar base, and original/transformed variables explicit. A marginal or conditioned calculation does not silently establish the joint operational object.
Pair cancellation with a nonzero-survival certificate	Reasoning pp. 26, 41–45; Ten Proofs pp. 126–139, 164–180, 204–212	A Möbius, parity, log-product, or represented-sum cancellation must prove that required terms survive and denominators remain valid. Equality after every useful term cancels is vacuous.
Pair relaxations, quotients, truncations, and regularizations with a proved map back	Reasoning pp. 7–10, 25–32, 34, 44; Ten Proofs pp. 13–17, 53–58, 126–151, 184–218, 238–242	The enlarged or smoothed object must satisfy the original constraints or have an exact return map. Quantization, added noise, or a restricted lattice can change the pid-rs estimand.
State adjacent non-implications and retain the smallest falsifier	Reasoning pp. 7, 12, 25–34, 44; Ten Proofs pp. 79–112, 140–151, 184–218, 236–249	Record which nearby theorem, computational model, source arrow, or stronger quantifier does not follow. A counterexample kills only the implication it instantiates.
Turn computation into a finite certificate before an asymptotic or universal claim	Reasoning pp. 12–14, 28, 30–32, 49, 56; Ten Proofs pp. 36–40, 105–121, 164–180	Check exact finite objects, margins, rounding, and resource bounds first. A plot, one fixture, or high-precision agreement remains bounded evidence.

Several tempting transfers were explicitly rejected:

- No Mellin/Fourier, coding, expander, group, operator-algebra, quantum, algebraic-geometric, lattice-hardness, Bergman-kernel, Ramsey, or Shannon-capacity theorem was imported. Shared words such as information, entropy, conditioning, lattice, measure, cancellation, or witness do not establish a PID correspondence.
- A route that merely failed is not a counterexample. It is a negative search result until an exact witness falsifies a named quantified statement.
- One witness for each object is not one simultaneous witness; one good benchmark cell is not one implementation valid on every declared cell.
- Bounded enumeration, sampled parameters, finite fixtures, one subsequence, or an asymptotic construction is not universal or all-index evidence without the stated bridge.
- Existence from a probabilistic argument is not an executable generator; a theorem statement or bibliography is not a checked application; and two bounds sharing an imported crux are not two independent confirmations.
- The collections' subject-matter conclusions, reported chains of thought, model confidence, and retrospective narrative are not pid-rs evidence.

The accepted controls influenced the claim schema, dependency-cut accounting, theorem-application map, finite-certificate path, exceptional-case checklist, holdout rules, and invalidation protocol below. They did not add a PID theorem, validate an estimator, close Programs A–E, or change a claim disposition. The table above is the retained compact source-review record. The





source PDFs, per-page renders, and scratch ledgers are not repository artifacts, so a replay must reacquire the exact-hash source bytes and regenerate its own page renders rather than treating the coverage statement as an embedded copy of the reviewed evidence.

3.5 AI model operating protocol

3.5.1 Applicability and risk

A **major claim** is any claim that changes a public method definition, theorem, estimator result, validated-status statement, release gate, or downstream readiness decision. A **high-consequence claim** is a major claim that could materially affect scientific inference, safety monitoring, mission or authorization policy, or a silent false numerical result. Treat an uncertain classification as the higher class until the claim packet justifies a downgrade.

Major claims require recorded role overlap, a claim packet, a counterexample route, the 20-lens adversarial audit below, and an explicit independence vector for every route pair. Do not compress functional/mechanism, epistemic/dependency, and institutional/custody independence into the word “independent.” High-consequence claims additionally require a dependency-disjoint epistemic route at every load-bearing shared bridge, specialist human review, and an explicit consumer no-go condition while any applicable gate is open.

3.5.2 Separate roles

Use explicit roles and deliverables for a major claim. Do not ask one undifferentiated run to set the problem, prove it, design its test, and adjudicate its own work.

Role	Required output	Invalid shortcut
Specification editor	Frozen claim packet and ambiguity table	Solving before the claim is fixed
Source checker	Immutable source and known-result map	Trusting the prompt premise
Proof route	Named subclaims and dependency graph	Confidence language as proof
Counterexample route	Exact falsifiers and boundary cases	Only random examples
Formalization route	Prose-to-formal object map	Silent weaker surrogate
Certificate route	Exact or interval certificate format	Opaque decimal oracle
Implementation route	Specification-to-code correspondence	Unit tests as refinement proof
Statistical route	Frozen calibration or holdout protocol	Development data as holdout data
Adversarial auditor	Attack log and unresolved objections	Restating the candidate proof
Integrator	Evidence matrix and scoped decision	Majority vote among models

One worker can fill more than one role only when the overlap is recorded. A final auditor must reconstruct the claim from the frozen packet and evidence. It must not rely only on the proof author’s summary. Role labels, separate contexts, fresh sessions, and adversarial prompts provide structured review only; they do not create institutional or model-lineage independence.

3.5.3 Independence vector

For each pair of evidence routes record the three-component vector (functional/mechanism, epistemic/dependency, institutional/custody) rather than one Boolean:





Component	Positive evidence	What does not establish it
Functional/mechanism	Different languages, arithmetic libraries, algorithms, data structures, or independently specified encodings that can expose route-specific defects	A renamed wrapper or the same generated table read twice
Epistemic/dependency	Materially different mathematical ideas with no shared unproved bridge, source ambiguity, oracle, formalization seam, or selected stopping rule at the claimed cut; a different model lineage can improve diversity but does not guarantee this	Fresh prose, a new prompt, or two derivations sharing the same imported crux
Institutional/custody	Separately controlled authority, access, and refusal capability: a person, organization, protected system, or custodian that cannot silently revise the original evidence or acceptance record	Different model lineage alone, one Codex-like agent in separate contexts, role labels, local hashes, or same-lineage review

A same-agent Rust/MPFR producer and Python exact-arithmetic checker can be functionally diverse and valuable. A same-lineage source-blind derivation can reduce one epistemic dependency. Neither is institutionally independent merely because it ran separately. State unknown or absent components instead of promoting a partial vector to “independent verification.”

3.5.4 Independent routes

For each major claim:

1. Give at least two routes the same frozen claim packet without the other route’s answer.
2. Require each route to state its starting point, assumptions, strongest result, exceptional cases, and likely failure mode.
3. Record a route memo before routes exchange results.
4. Share route artifacts and stated implications, not hidden reasoning traces.
5. Identify shared unproved lemmas, source material, generated data, and oracles.
6. Count routes that use the same unproved bridge as one route for confidence purposes.
7. Preserve every materially distinct valid proof or solution as a separately named route, even after selecting a preferred exposition. Do not overwrite a special-case proof with a later generalization or merge two mechanisms merely because they reach the same conclusion.

Prefer method diversity to model-name diversity. Examples are a coupling proof and exact enumeration, a real-analysis proof and a formal kernel, or a generator and an independently written checker. Repeated passes by the same model against the same prompt, source text, imported theorem, or proposed proof are correlated evidence, even when they occur in fresh sessions or use harder reasoning settings. They can reveal instability and generate attacks, but they do not satisfy the epistemic route requirement unless their load-bearing dependencies are genuinely disjoint, and they never establish institutional/model-lineage independence by session or role label alone.





3.5.5 Critical-cut-set accounting

Count independence at the level of proof dependencies, not prose narratives. Represent obligations as an AND/OR acyclic hypergraph: every tail of an AND-hyperedge is required, while alternative incoming hyperedges are OR-routes; logical alternativity alone does not make them dependency-disjoint. A node closes only when one permitted incoming hyperedge has all required premises closed and its implication checked. Define each route as a set of dependency nodes in a frozen admissible vertex universe. Unless a deliberately trivial cut is reported separately, that universe excludes the common claim/goal node and synthetic route or AND/OR aggregator nodes. Record every inclusion-minimal shared cut set: each inclusion-minimal admissible node set that intersects every route in the declared route family. State whether the family is candidate, permitted, or accepted, and tag each cut node as open, closed, or a known common cause. An accepted route cannot contain an open node; a cut over accepted routes may nevertheless expose a closed shared dependency.

For the worked two-route figure, freeze $U = \{A1, A2, B1, C\}$, $R_A = \{A1, A2, C\}$, and $R_B = \{B1, C\}$. A cut is a set $H \subseteq U$ with $H \cap R_A \neq \emptyset$ and $H \cap R_B \neq \emptyset$. The complete inclusion-minimal cut family is $\{\{C\}, \{A1, B1\}, \{A2, B1\}\}$. This follows without trusting the drawing: if $C \in H$, minimality forces $H = \{C\}$; if $C \notin H$, hitting R_B forces $B1 \in H$, and hitting R_A then forces exactly one of $A1$ or $A2$. The figure metadata records these same sets, and the publication checker independently enumerates their finite transversal family. Omitting $\{A2, B1\}$, or retaining a nonminimal superset, is a failing semantic mutation.

Two derivations that use different case splits but obtain their power from the same imported theorem, unproved lemma, generated table, numerical oracle, or implementation are correlated at that cut set. They can cross-check local algebra outside the cut set, but they do not independently close it. A route summary must therefore include:

- the critical cut set;
- all other routes that share each cut-set node;
- the evidence type that closes each node;
- whether the node was re-derived, checked against a primary source, assumed, or only restated;
- one falsification attempt directed at each shared node.

Use a dependency-disjoint route, not another paraphrase, when a high-consequence claim depends on one shared bridge. Examples include an exact enumerator versus a symbolic proof, a Python integer/Fraction checker versus a Rust MPFR producer, or a formal theorem versus a numerical fixture.

3.5.6 Frozen run context

Record this full context for each model run promoted to evidence or used to adjudicate a claim:

- claim ID and revision;
- pid-rs commit;
- paper, generated document, and formal artifact revisions;
- proof and imported-library toolchains;





- Rust compiler, `Cargo.lock`, target, and feature set when code is in scope;
- generated table and lattice digests;
- exact definitions, conventions, assumptions, and non-solutions;
- ex-ante intended closure and falsification routes, the initial accepted-result label, and completion checks;
- prompt text, model identity, run date, output path, and output digest.

For a non-exploratory run that is not promoted or adjudicating, retain only enough identity, purpose, disposition, and reopen condition to prevent accidental reuse or double counting. Retain the full context above and decisive output for every evidence-promoted or adjudicating run. Summarize exploratory failures with their route, falsifier, and reopen condition; do not mistake a digest for proof or archive every routine prompt merely to increase evidence volume. Hidden reasoning traces are neither required nor an accepted evidence artifact.

Without this context, treat the output as exploratory. It cannot change a claim disposition.

3.5.7 Freeze purpose and identity boundary

Do not use “frozen” without naming its object and purpose. For every freeze record: the exact object; the threatened error; the checker, CI gate, release system, or custodian that refuses to proceed on mismatch; whether the same actor can revise both object and expected record; and whether the theorem or statistical design actually requires fixedness.

Freeze class	Object and purpose	Required refusal point and limitation
Mathematical claim	Definitions, types, quantifiers, premises, conclusion, and non-solutions; prevents proof of a nearby statement	Claim revision and review gate refuse silent semantic drift. A same-author revision is visible, not independently authenticated.
Exact bytes	A named raw file, canonical payload, source tree, certificate, release asset, or external source; prevents stale or substituted instance use	The consuming checker or release gate recomputes identity and refuses mismatch. A colocated digest detects drift but is not external authentication.
Fitted transform	Parameters and behavior of a measurable transform trained only from declared fit information; prevents evaluation leakage or adaptive remapping	Evaluation code refuses an unbound/refitted transform. Fixedness may be a theorem premise; its digest only identifies the represented transform.
Analysis/holdout plan	Metrics, tolerances, multiplicity, failures, seeds/target custody, stopping, and acceptance rules; reduces result-dependent tuning	Adjudication refuses deviations or records a new version. Call it blind only when inaccessibility is enforced; otherwise it is controlled.

Git-tree identity, SHA-256 of raw file bytes, canonical mathematical-object identity, and external authentication answer different questions. A Git commit binds a tree and paths; a raw-file digest binds one encoding; a domain-separated canonical digest binds the declared mathematical serialization; a signature, protected log, or independent custodian can authenticate or anchor an identity. None proves the object’s mathematical correctness. Retain extra digests where they bind cross-artifact, release, package, certificate/input, or external-source bytes, or where mechanism diversity checks an encoding. Do not multiply routine prose digests and call the count evidence.





3.5.8 Durable agent continuity

Before a long-running command, after every consequential result, at every observable checkpoint before a possible context boundary, and immediately after detecting compaction or resume, write or update a durable continuity record outside secret material. Current Codex surfaces do not expose a guaranteed pre-compaction hook; a future harness may add one, but this protocol must not claim that an unobservable checkpoint was recorded. The record must contain:

- objective plus the exact permitted normative user prompt and instruction locators; when the prompt contains protected spans, store a redacted canonical projection plus immutable locators and an identity of the permitted projection, while representing protected spans only by custody locators or permitted opaque identities;
- exact in-scope work and explicit non-solutions;
- repository path, remote URL, HEAD commit, tree identity, worktree path, clean/dirty state, and any alternate-index identity;
- owner for every agent, worktree, mutable path, immutable path, and external evidence location;
- each running command with session identifier; PID and process-group ID only when the tool exposes them or they are reliably observed, otherwise explicit **unavailable**; evidence directory, expected outputs, and a safe non-destructive poll rule;
- primary-source and citation identities, versions, locators, and locally checked byte digests when the downloaded bytes matter;
- every closed gate with its exact evidence and explicit non-implications;
- open obligations, blockers, dependencies, and every invalidated or zero-credit run with cause;
- user decisions, permissions, and forbidden actions;
- the next safe read-only or in-scope action; and
- record schema version and UTC update time as observation metadata, never as preregistration or time authority; when a continuity transport needs byte identity, place raw-byte SHA-256 in an adjacent handoff/sidecar or the next resume record, or define an explicit canonical projection that excludes the identity field, and state which method was used. A matching colocated record and digest provides drift detection, not authentication.

Resume protocol:

1. Read the normative prompt, applicable instructions, and continuity record fully.
2. Verify live Git/remote/worktree state, processes, and named files with read-only checks.
3. Reconcile every drift. The continuity record is a memory aid, not authority, attestation, or theorem evidence; a colocated digest does not authenticate it.
4. Do not restart closed milestones, double-count prior evidence, transfer credit across a changed claim/fixture/toolchain, or silently reuse a zero-credit run.
5. Revalidate external facts that are missing, stale, mutable, or outside the pinned source scope.





6. Update the record at the next observable checkpoint before another possible context boundary or long run, immediately after a consequential pass, failure, invalidation, permission change, or scope change, and immediately after detecting compaction or resume.

Never store credentials, tokens, decryption keys, secret holdout rows/seeds/targets, hidden reasoning traces, or other protected material in the continuity record. Store only custody locators and permitted opaque identities for inaccessible assets.

Compact copyable template:

```
CONTINUITY v1 | updated_utc: <observation metadata, not time authority>
record_identity_method: <sidecar/next-record raw SHA-256, or projection excluding this field>
objective: <exact objective>
normative_prompt: <exact permitted prompt, or redacted canonical projection>
normative_locators: <prompt/instruction paths or immutable message IDs; protected custody
  ↳ locators>
scope: <in scope> | non_solutions: <explicit exclusions>
repo: <path> | remote: <url> | HEAD: <commit> | tree: <tree>
worktree: <path> | dirty: <exact status> | alternate_index: <none or identity>
ownership: <agent/worktree/path/evidence owners; immutable paths>
running: <command; session; PID/PGID or explicit unavailable; evidence_dir; safe_poll; or
  ↳ none>
sources: <citation/version/locator/raw-byte identity as applicable>
closed: <gate -> exact evidence -> non-implications>
open: <obligation/blocker/dependency>
invalid_zero_credit: <run/artifact -> cause>
decisions_permissions_forbidden: <user decisions; allowed and forbidden actions>
next_safe_action: <one exact action>
```

3.5.9 Codex goal, plan, and tool lifecycle

A long-running Codex execution has three distinct state layers. The **scientific claim state** is the authority for mathematical completion; the **goal state** records the user's unfinished objective; and the **plan state** is only a mutable scheduling aid. A token count, elapsed time, model confidence, completed plan step, or the mere success status of a tool call is not evidence of scientific progress. Validated and retained tool output may supply evidence for the exact obligation it checks.

Under the current Codex function contract, apply this lifecycle:

1. Create a durable goal only when the user explicitly requests one. Use the exact objective, and set a token budget only when the user explicitly supplied a budget. Do not replace an unfinished goal with a narrower convenience goal.
2. Call `get_goal` after a resume or compaction, before a progress/completion report, and before a terminal goal transition. Reconcile it with the continuity record and live repository evidence; neither source silently overrides the other.
3. Use `update_plan` to expose sequencing, with at most one step in progress. A plan item closes only when its stated work is done, but this still gives no evidence credit unless its claim gate also closes.
4. Use `update_goal(status="complete")` only when the exact objective is achieved and no required work remains. Near-exhausted compute or a convenient stopping point is not completion. Use `blocked` only after the same impasse has recurred for at least three consecutive goal turns, counting the original user-triggered turn, and no safe in-scope progress remains; record the blocker and each attempted alternative. When a previously





blocked goal resumes, begin a fresh three-turn blocked audit. A paused, hard, slow, or partially complete goal stays active.

5. Derive progress from a versioned obligation registry, not from tool usage. Publish the weighting rule, count only closed applicable obligations, retain failed and unknown states separately, and report a range when the denominator or scope can still expand.

Some Codex surfaces may spell goal creation as `create_goal` and a user may call the action “set goal.” A harness adapter must bind the semantic operation and verify the returned goal identity/status; it must not pretend an unavailable call succeeded. Goal-service state is coordination metadata, not proof, archive custody, or authentication.

3.5.10 Tool-call and agent receipts

For every consequential tool action, classify it before execution as read-only, reversible in-scope mutation, external state change, or destructive. Record the exact subject and authority. A replayable receipt contains, as applicable:

- tool/function name and contract revision or client version;
- domain-separated operation ID, goal ID, claim ID/revision, and dependency IDs;
- exact repository root, worktree, HEAD, tree, dirty-state projection, and alternate index;
- normalized argument projection, working directory, relevant environment projection, toolchain, platform, and input artifact identities;
- start/end observations, exit status, stdout/stderr identities, produced artifact identities, and whether output was truncated;
- expected-versus-observed predicate and explicit non-implications; and
- failure disposition, retry policy, cancellation state, and safe resume/poll identifier.

Do not record secrets or hidden reasoning. A timestamp orders observations only when its clock and authority are declared; it is not a cryptographic time proof. A receipt produced and checked by the same mutable authority detects accidental drift but is not independent attestation.

For a long command, retain the returned session/cell identifier and poll with the corresponding non-destructive wait function; do not launch a duplicate merely because no new output arrived. Before retrying, determine whether the prior process is running, terminal, or externally unobservable. Bind the final result to the original command rather than reporting only the last poll.

Delegate only bounded tasks with an explicit input subject, mutable-path ownership, forbidden actions, deliverable, and completion test. Two agents must not edit the same mutable path concurrently. Use read-only hostile reviewers for frozen candidates, and record whether their starting assumptions, implementations, source access, model lineage, and custodians actually differ. Codex subagents can add functional or epistemic diversity; being subagents does not by itself add institutional independence.





3.5.11 Publication-harness state machine

A future executable harness should make the workflow below a typed acyclic graph, not infer it from prose. Each immutable claim revision owns:

- a premise registry and positive mapping obligations;
- artifact nodes for source, formal statement, certificate, generator, checker, executable, data, statistical plan/result, rendered document, package, and hosted receipt;
- gate nodes with **open**, **passed**, **failed**, **blocked**, or reasoned **not_applicable** state;
- directed edges labelled **AND**, **OR**, **derives**, **checks**, **renders**, **packages**, or **observes**, with every OR branch preserving its own premises;
- exact evidence identities, route-independence vectors, negative controls, limitations, and downstream non-implications; and
- a decision node that refuses completion while an applicable required gate is not passed.

Orient every dependency-bearing edge from prerequisite to dependent. A semantic change to a premise, definition, generator, checker, compiler/toolchain assumption, fixture, threshold, or mapping theorem invalidates every semantically dependent result reachable from that object. A raw byte change separately reopens every exact-byte-dependent gate that binds those bytes. It does not, by itself, falsify a mathematical theorem whose dependency-disjoint route is unchanged. Such a route may remain passed only when the graph contains a separately checked impact/equivalence map showing that no changed semantic prerequisite can reach any node used by the route. Formatting or serialization changes therefore still require artifact replay, while mathematical credit survives only through an explicit semantic-equivalence boundary. The harness moves affected descendants back to **open** or **blocked**, retains the old receipt as historical evidence, and requires the appropriate replay. This is the mechanism for correcting earlier work when a later finding truly invalidates it without pretending that every changed byte changes every theorem.

Hosted execution and self-binding artifacts require an acyclic commit protocol: commit and push the subject; wait for its exact hosted run to become terminal; then commit a descendant receipt that binds the subject and run. The receipt commit cannot contain evidence about its own future hosted run. If that receipt itself needs hosted observation, keep it external or bind it in a later, explicitly scoped descendant - never claim a self-hash or infinite receipt chain.

For publication, the harness must require exact agreement among canonical Markdown, any embedded typesetting source, generated tables/figures, and the committed PDF; deterministic rebuild where claimed; warning/font/link/metadata checks; extracted-text and page-geometry comparison across the declared toolchains; and page-by-page rendered visual review. A PDF that is newer in prose but stale in bytes, or byte-current but visually defective, fails the publication gate. Machine and human artifacts must expose the same claim revision, evidence state, limitations, and open obligations.

Compact harness transition rule:





```

local_closure(node) :=
  (leaf(node) AND accepted_evidence_of_declared_class(node))
OR
  (internal(node)
   AND exists permitted incoming hyperedge e -> node
   AND every tail node of e is accepted
   AND implication_of(e) is discharged)

accept(node) :=
  schema_valid(node)
  AND exact_subject_identity(node)
  AND local_closure(node)
  AND every_positive_mapping_edge_discharged(node)
  AND negative_controls_fail_closed(node)
  AND limitations_and_non_implications_present(node)
  AND no_reachable_invalidated_dependency(node)

publish(claim_revision) :=
  all_applicable_required_gates_passed
  AND decision_receipt_acyclic
  AND machine_human_artifacts_coherent
  AND release_subject_matches_reviewed_subject

```

These predicates specify refusal behavior; they do not prove the mathematical predicates embedded inside a node. Each mathematical premise still needs its named proof, certified computation, accepted empirical design, or explicit assumption status.

3.5.12 Evidence labels and route memos

The following are **artifact-verification labels** for substantive model statements. They are not the accepted evidence classes or the claim dispositions defined in the acceptance rule. Use one:

- UNVERIFIED-MODEL-OUTPUT;
- CHECKED-SYMBOLICALLY;
- CHECKED-EXACTLY;
- FORMALLY-CHECKED;
- CERTIFIED-NUMERICALLY;
- IMPLEMENTATION-REFINED;
- EMPIRICALLY-CALIBRATED;
- CONSUMER-QUALIFIED; or
- REJECTED-BY-COUNTEREXAMPLE.

Each route memo must record:





```

Route ID:
Claim revision:
Mathematical family:
Starting point and independence vector:
Current obligation:
Strongest established result:
Exact evidence:
Counterexamples attempted:
Exceptional cases:
Missing lemma or bridge:
State:
Reopen condition:
Artifact paths and digests:

```

A route that reduces the target to an unproved claim of comparable strength is blocked, not complete.

Keep three fields separate in every claim schema:

Field	When fixed or appended	Meaning
Artifact-verification label	Appended to one reviewed artifact	What kind of check that artifact actually survived; it is not the claim's evidence state
Intended closure/falsification routes	Frozen ex ante in the claim packet	Which premise-explicit proof, certificate, test, holdout, or counterexample routes are permitted to close or falsify each obligation
Current accepted-result evidence records	Initialized as an empty list rendered as the literal text "no accepted evidence"; append-only after adjudication	Zero or more records, each with exactly one evidence class, scope, assumptions, artifact identities, adjudication, and invalidation/supersession state; the claim disposition remains separate

A diagnostic may create an obligation or motivate a falsifier, but it does not silently become a closure route. A counterexample closes only the exact negative implication or disproof whose quantifiers it instantiates. Do not rewrite the ex-ante field after seeing a favorable result; create a claim revision instead.

3.6 pid-rs protocol

The rest of this note defines a repository protocol. It is an adaptation, not a source claim.

3.6.1 1. Write an exact-claim packet

Create one packet before work starts. Give the packet a stable claim ID. Include these fields:

Field	Required content
Claim kind	Exactly one primary kind: definition/semantic identity; mathematical theorem; statistical/estimator performance; formal correspondence or certificate; executable conformance; custody/release qualification; consumer/downstream readiness; or another precisely defined kind. Dependencies on another kind become separate typed obligations or claim IDs.





Field	Required content
Claim	One versioned, type-specific proposition or specification-conformance statement with one disposition. Split conjunctions whenever components can close, fail, expire, or be invalidated separately.
Objects	Every exact object used by the selected kind: mathematical domains, codomains, alphabets, measures, lattices, sample spaces, and source/target roles; formal statements and encodings; executable APIs, artifacts, builds, and platforms; custody/release subjects; and consumer system, use case, authority, and decision boundary. Mark an inapplicable object class with a typed reason.
Support/reference measure	Population support, sigma-algebras, dominating/reference measures, densities or RN derivatives, boundary and singular cases
Lattice/source count	Exact source count, finite antichain carrier, typed redundancy order, event map, and zeta/Möbius direction
Quantifiers	Their full order, including every uniformity requirement
Sampling	Row law, independence/dependence class, conditioning sigma-field, stationarity/ergodicity or named coefficients/rates, splits, and sample size
Assumptions	A typed premise ledger assigning every premise to the exact object and downstream edge that uses it
Units/gauge	Logarithm base, units, metric, scale, preprocessing/measurement gauge, and which changes alter the estimand
Mapping obligations	Every cross-definition, discrete/continuous, transformed/untransformed, paper/code, or formal/prose transfer and the positive theorem needed to justify it
Representation	Exact-real, symbolic, interval, high-precision reference, binary64, serialized, compiled, and wrapper claims kept distinct
Conclusion	The exact versioned definition; equality, inequality, quantified theorem, limit, coverage, calibration, error, or abstention statement; formal-correspondence claim; executable refinement/conformance property; archive/release predicate; or bounded consumer readiness/no-go decision appropriate to the selected kind, including scope and non-implications
Non-solutions	Weaker statements that do not complete the claim
Falsifiers	Boundary cases and counterexamples that can refute the claim
Intended closure/falsification routes	Frozen ex ante for each obligation: premise-explicit mathematical proof, machine-checked proof, certified computation, scoped test or holdout, and a counterexample route where falsification is possible
Initial accepted-result evidence records	Empty, rendered as the literal text “no accepted evidence”; later adjudicated records are appended one class per record and never substituted for the intended-route field
Completion predicate and adjudicator	A fail-closed predicate, mechanical where possible, plus the named human authority where required, that checks whether every applicable typed obligation is already closed by current accepted evidence and whether no unresolved falsifier, contradiction, expiry, or invalidation remains. It may adjudicate the disposition; it cannot create, replace, or upgrade missing evidence.





Do not overwrite a claim packet after you inspect a result. To correct or change it, create a new revision and retain the old revision.

Retain the full packet schema for every claim kind. Mark a field `not_applicable` only with a typed reason showing why the claim has no corresponding mathematical, statistical, formal, executable, release, or consumer obligation. A reasoned non-applicability record closes only that applicability question; it supplies no evidence for any other field.

Completeness is a disposition, not an evidence class. A green completion checker is evidence only for the exact checker-conformance claim it exercised. It does not by itself close a mathematical, statistical, formal, executable, release, custody, or consumer claim. Each typed obligation still needs an accepted record from one of its frozen permitted routes, with every mapping edge closed.

Add a semantic pin for every ambiguity that changes the mathematical object. Quote or transcribe the controlling primary-source statement, record the competing readings, choose one reading with a reason, and state which downstream obligations depend on it. An implementation test cannot repair a proof about the wrong object.

No similarity, limiting intuition, unit agreement, shared atom names, or successful API call is a mapping theorem. Before transferring a result, state a positive theorem whose domain, codomain, premises, map, preserved property, and conclusion match the claim packet. If that bridge is absent, mark the edge `OPEN` or `BLOCKED` and abstain from the transfer. Keep empirical-versus-population and exact-real-versus-binary64 obligations on separate nodes even when the formulas look identical.

3.6.2 2. Build an obligation graph

Split the claim into named obligations. Each edge must state why one obligation implies its destination obligation. Use separate nodes for these classes:

- definition and semantics;
- mathematical reduction;
- finite algebra;
- analytic inequality or limit;
- numerical certificate;
- estimator behavior;
- software conformance;
- statistical validation;
- downstream acceptance.

If an edge depends on an unproved lemma, create a separate obligation node. Mark it as open until evidence closes it.





3.6.3 3. Keep a separate-approach registry

Use one row for each mathematical idea. Do not use one row for each worker or prompt.

Field	Meaning
Approach ID	Stable identifier
Family	Main mathematical mechanism
Starting inputs and independence vector	Ideas not copied from another route; functional, dependency, and custody overlaps stated explicitly
Current obligation	The exact node under study
Best result	Strongest proved statement
Counterexample search	Cases that were tested
Missing lemma	Exact open step
State	Proposed, active, blocked, falsified, merged, or complete
Reopen condition	New fact that can justify more work
Artifact links	Notes, code, proof files, fixtures, and logs

For PID work, start with different families when they apply. Useful families include Möbius and lattice algebra, measure-theoretic analysis, concentration inequalities, information geometry, optimization, exact finite enumeration, and certified interval analysis.

Do not give every route the current preferred answer. First, let each route state its own assumptions and failure modes. Combine routes only after this step.

3.6.4 4. Retain blockers and failed routes

Record every failed route. Keep its exact statement and its failure reason. Use one of these failure classes:

- a concrete counterexample;
- a false intermediate lemma;
- a missing assumption;
- a quantifier error;
- a reduction to a claim of comparable strength;
- a numerical certificate that does not cover the full domain;
- a machine-checked proof that checks only a weaker statement;
- a statistical design that reuses development data.

Do not delete an invalidated derivation. Mark it clearly as invalid. Add a small counterexample when one is available. State why it falsifies the route. A route can reopen only when new retained evidence resolves its recorded failure without silently weakening the claim. That evidence may be a corrected lemma, completed bridge, repaired certificate or implementation, a new mechanism, or a stronger premise that the claim packet permits.





3.6.5 Load-bearing correction ledger

When an audit finds an error, do not classify it informally as “minor” or “fatal.” Trace it through the obligation graph. Record:

Field	Required content
Error	The exact false statement, code path, or certificate field
Detector	Counterexample, invariant, proof checker, mutation, or source comparison
Smallest witness	Minimal retained input that exposes the error
Dependency reach	Every claim node reachable from the false node
Load-bearing status	Whether any permitted conclusion depends on that node
Correction	New statement or implementation and why it is valid
Regression	A test, theorem, or mutation that fails before and passes after the correction
Residual boundary	What the correction still does not establish
Artifact revisions	Old and new claim, proof, source, and evidence digests

A non-load-bearing error still remains in the ledger. “The conclusion survives” is acceptable only after the dependency reach is explicit and a separately checked, dependency-disjoint accepted path establishes the conclusion without the false node.

Likewise, a local counterexample refutes only the quantified statement or implication edge it instantiates. It does not refute a downstream theorem when an alternative proof route remains open; trace the OR/AND dependency graph, invalidate every route that uses the false edge, and leave the target **OPEN** or **BLOCKED** until another route is either proved or refuted. Do not promote “this proof fails” to “the theorem is false.”

If a result expands from a special case to a uniform or higher-dimensional theorem, create a new claim revision. Preserve the narrower proof and its evidence. Re-open all obligations involving new quantifiers, uniform constants, exceptional families, or imported theorems. Agreement on the old special case does not validate the generalization.

3.6.6 5. Convert computation into certificates

Use numerical work to find structure. Do not treat a plot or a floating-point match as a proof. Use this conversion path when it applies:

```
exploratory computation
-> exact reduction
-> rational or outward-rounded interval data
-> finite certificate statement
-> machine check
-> separately implemented replay with a recorded independence vector
```

Keep the generator, its inputs, the generated certificate, the checker, and their digests. Add documented negative mutations. The checker must reject each mutation.

Rational probabilities do not imply rational logarithmic information values. Use a symbolic identity or a certified interval for a proof claim. Label an uncertified high-precision value as a reference. Do not call a decimal reference an exact entropy oracle.





3.6.7 6. Run the required 20-lens adversarial audit

Complete one applicability matrix for every major claim and every live artifact that states or transfers it. Every row receives exactly one disposition: **PASS** with evidence, **CORRECT** with the old and corrected statement, **NARROW** with the retained scope, **OPEN** with an obligation or blocker, or **NEGATIVE** with a retained counterexample. N/A is allowed only with a typed reason. If one lens exposes separable sub-obligations with different dispositions, split that lens into typed subrows; never collapse them to the most favorable disposition. A paragraph saying that an audit occurred is not a completed matrix.

Lenses 1–10: scientific object and inference contract

Lens	Required applicability question
1. Estimand	Is the exact functional, estimator, transformed estimand, or diagnostic named without substitution, and is the order of conditioning, fitting, mixing or averaging, nonlinear transforms, and Möbius inversion fixed?
2. Types	Are domain, codomain, variable roles, data representation, and output type explicit?
3. Quantifiers	Is the full order of existence, universality, limits, uniformity, probability, and conditioning preserved?
4. Mappings	Is every transfer backed by a positive, premise-checked mapping theorem, including one compatible joint version or witness when simultaneity is required; otherwise does the route abstain?
5. Support/reference measure	Are population support, dominating/reference measures, density or RN premises, boundaries, and singular cases declared?
6. Lattice/source count	Is the exact finite antichain poset, order, source count, event semantics, and Möbius direction fixed?
7. Units/gauge	Are logarithm base, units, metric, scale, preprocessing gauge, and conversion limits explicit?
8. Population/empirical	Are population law, empirical PMF, estimator, fixture, and represented output kept distinct?
9. Sampling	Are row law, i.i.d./stationary/ergodic/dependence coefficients, rates, splits, and conditioning stated?
10. Selection/UQ	Are fitting, tuning, reuse, multiplicity, null, coverage, and abstention contracts justified without calibration transfer?

Lenses 11–20: evidence, custody, implementation, and release contract

Lens	Required applicability question
11. Formal correspondence	Does a reviewed prose-to-formal map bind the actual objects and conclusion, not a convenient surrogate?
12. Kernel/axioms/toolchain	Are exact versions, trusted kernel, imported axioms, <code>--trust=0</code> or equivalent, and unformalized bridges inventoried?
13. Route dependency diversity	Is the functional/epistemic/institutional independence vector stated, with shared cut sets and common causes?





Lens	Required applicability question
14. Numerical/binary64	Are exact-real, high-precision, interval, and binary64 claims separated with bounds for rounding, cancellation, overflow, underflow/subnormals, signed zero, NaN/ $\pm$ Inf propagation, rounding mode and FMA contraction, platform <code>libm</code> , and compiler/target variation; and is one total error budget smaller than every strict claimed margin?
15. Compiled/wrapper parity	Is every claimed Rust feature/build path, backend, Python wrapper, serialization, and failure surface checked?
16. Counterexample/mutation	Are boundary falsifiers, malformed fixtures, wrong mappings, weakened premises, and optimized-mode mutations required to fail closed?
17. Custody/threat/refusal	For each freeze, are the object, error threat, enforcement refusal point, same-actor limit, and theorem need recorded?
18. Citations/novelty	Are immutable primary sources, exact imported statements, application hypotheses, provenance class, and no-novelty boundary correct?
19. Ecosystem/authority	Are consumer snapshots, authority direction, capability gaps, acceptance status, and stale derived projections explicit?
20. Resource/platform/release	Are resource ceilings, cancellation, determinism, OS/architecture/toolchain scope, package identity, release assets, and non-implications closed?

For every strict sign, ranking, or separation claim, bind an exact or enclosed positive margin and one aggregate error budget covering every applicable approximation, tail, dependence, finite-range, representation, and compilation contribution. Componentwise bounds do not close the claim unless their joint composition is proved. If the aggregate budget can reach the margin, report the result as unresolved rather than selecting the favorable sign or ordering.

An audit must try to find a counterexample, a violated premise, a wrong mapping, or a numerical failure. It must not only restate the proof. Record the challenge, disposition, exact evidence, non-implications, and revision. If the revision changes an assumption, create a new claim-packet revision. Apply this matrix retrospectively: an older green gate or polished paper is not grandfathered.

SxPID definition-compatibility firewall

The routing problem begins with multi-source PID. Within the finite MGW/shared-exclusions construction used here, for any one-element antichain $\{a\}$ whose member a is a nonempty source-index subset, the cumulative self-redundancy equals local mutual information for the joint source block S_a and averages to $I(S_a; T)$. This is a cumulative-node identity: it neither says that the Möbius atom at $\{a\}$ equals mutual information nor adds another atom to the lattice. This statement is not transferred to every PID proposal. PID is not one uniquely defined functional, and shared exclusions itself has typed constructions that must not be collapsed. MGW is the finite-PMF pointwise construction implemented by the stable categorical code. Schick-Poland proposes an auxiliary-indicator/RCP/RN construction for a finite source family in its stated locally compact Radon/Borel setting. At $P(R) > 0$, the standard conditional probability is $P(T \in A \mid R) = P(T \in A, R)/P(R)$, which supplies the normalization needed for the MGW specialization. The reviewed arXiv v2 display writes only the numerator, so that display by itself does not establish the claimed recovery without a correction or clarified definition. Stable pid-rs code computes MGW and provides no separate general Schick-Poland





route. Its reference-measure/disintegration argument additionally invokes Standard Borel and countable-generation/separability machinery whose bridge from the displayed topology/measure clauses requires an explicit theorem or sufficient full-measure reduction, and target-local RN derivatives are only a.e. representatives without a selection rule even on Euclidean spaces. Eventwise RN derivatives need a simultaneous kernel/version theorem, a general source variable has no pointwise inverse, a Borel isomorphism alone does not supply the atom-plus-Lebesgue density representation displayed with Corollary 3.1 (physical PDF page 9; a Cantor law is a singular-continuous diagnostic), and a bimeasurable bijection need not be bicontinuous. The reviewed bicontinuity step is in Section 4.3.3 (physical PDF page 16). For null indicator events, RCP uniqueness is also only almost everywhere, so pid-rs treats these standard-Borel/RN-representative and indicator-value selection/limit bridges as open rather than universally validated rules. Ehrlich gives a practical, gauge-dependent continuous analytic density/quasi-density route with a source-disjunction kNN neighbourhood estimator inspired by, but not identical to, Schick-Poland and default-off here. The disjunction is not a probability-union law, and its quasi-density need not integrate to one. Ehrlich Definition 1 and the explicit non-normalization discussion are on physical PDF page 5. Appendix B derives a logical-statement quasi-density and includes mixed discrete/continuous examples; Appendix K begins on physical PDF page 27 and separately sketches a mixed-system treatment/estimation ansatz and one symbolic example, but no demonstrated or calibrated mixed estimator. Neither is the continuous PID3 branch-dimension problem, and pid-rs implements no generally calibrated mixed estimator. A theorem, fixture, estimator result, atom sign, or calibration statement from another PID or another construction must not be used as evidence merely because both methods use the words redundancy, unique information, and synergy. Before importing a PID result, bind all of the following:

1. the exact measure and immutable definition revision;
2. the target, ordered sources, source collections, and antichain order;
3. the construction-native local or target-outcome-specific information object, any cumulative-event semantics and Möbius convention it actually uses, and a typed `not_applicable` for absent fields; for MGW, bind the informative, misinformative, and signed-net terms explicitly;
4. the probability domain, estimator, preprocessing map, units, and numerical representation;
5. the source theorem's hypotheses, including every positivity or identity axiom; and
6. an explicit mapping theorem stating the property preserved by the transfer.

Shared atom names, the same three mutual-information coordinates, a similar benchmark value, or a common lattice are not a mapping theorem. In particular, Williams–Beer $I_{\min}$, BROJA, CCS, MMI, SID, and other authors' PID definitions are comparison objects unless this mapping is proved.

Two Lyu–Clark–Raviv works require separate theorem records. The published PRE article “Multivariate Partial Information Decomposition: Constructions, Inconsistencies, and Alternative Measures” (reviewed [arXiv:2508.05530v2](#)) assumes its stated Axioms 2–4, independent identity, and cross-subsystem reconstruction package. MGW's independent-COPY and signed-XOR values place it outside that premise package, so the published theorem neither refutes MGW nor is refuted by those values. The later “Structural Impossibility of Antichain-Lattice Partial Information Decomposition” ([arXiv:2604.03869v2](#)) instead stipulates recoverability-descriptor atoms; its impossibility transfers only to a PID that factors through that descriptor. Sharing an antichain carrier supplies neither premise package nor descriptor factorization.





The Schick-Poland finite-discrete specialization can map to MGW by standard conditioning when the supported event has positive probability and the required $/P(R)$ normalization is present; the reviewed arXiv v2 Section 4.3.1 display (physical PDF pages 13–14) omits that factor, so the displayed recovery remains an open source-level correction/clarification obligation. This does not close the general kernel, representative, invariance, or null-indicator obligations. Ehrlich’s matched refining-bin calculation is a narrower premise-bound asymptotic motivation from discrete inclusion-exclusion to a density expression; it is not identity of all three constructions, a general convergence proof, or a pid-rs fixed-quantizer convergence theorem. A discrete exact-real theorem still does not validate a continuous estimator or an unquantized estimand. A measure-independent theorem may be imported only after its abstract objects and all hypotheses are instantiated with the actual typed construction. Record failed or missing mappings as negative/open results and abstain rather than silently transplanting them.

Terminology firewall: “categorical SxPID” means finite/discrete category-valued random variables and a finite PMF. The population functional may use a declared finite population PMF, whereas the pid-rs direct row-data path computes $F(\hat{p}_n)$. That value is a descriptive empirical functional when the empirical law is the target and a plug-in estimator when the target is the population functional $F(p)$; binary64 representation and error remain separate. This usage is unrelated to category theory. Möbius inversion here is on a declared finite antichain poset. Category-theoretic machinery or infinite-poset inversion requires a separate typed theorem and cannot be inferred from the word “categorical.”

Imported-theorem application map

For every external theorem that carries a load-bearing step, retain an application record:

```
Source theorem and immutable version:
Exact source statement:
Named source arrow (domain -> codomain):
Property and direction imported (injective/surjective/isomorphic/zero/exact):
Local variables-to-source variables map:
Required hypotheses:
Evidence for each hypothesis:
Exceptional cases and exclusions:
Uniformity and constant dependence:
Named local arrow (domain -> codomain):
Arrow-level type and direction correspondence:
Source-language ambiguities and resolution evidence:
Conclusion actually imported:
Local obligation closed:
What was checked against the source:
What was not re-proved:
```

Checking that a theorem exists is not checking its application. Checking the application is not re-proving the source theorem. State both boundaries. In particular, audit whether an implied constant is uniform over the changing family used locally, whether an exception clause is exhaustive, and whether a transformed object remains in the source theorem’s domain.

Citation-edge type check

An imported statement containing more than one morphism, an exact sequence, a commutative diagram, or an anaphor such as “which,” “it,” or “this map” cannot close an obligation until the following gate passes:

1. Give every source morphism a stable local identifier and a typed signature, for example $f_s : A_s \rightarrow B_s$ and $g_s : B_s \rightarrow C_s$.





2. Bind every imported predicate to one named arrow and direction: for example `Mono(f_s)`, `Epi(g_s)`, `Iso(g_s)`, or `Zero(g_s)`. A free-floating “is an isomorphism” fails.
3. Name the local arrow and show the domain, codomain, variance, indexing, and direction match the source arrow under the recorded variable map.
4. Resolve ambiguous source prose from the upstream proof, adjacent lemmas, errata, or a specialist source check. If competing readings remain, retain both and mark the obligation `BLOCKED`; do not choose the reading that makes the local proof work.
5. Re-derive the local implication from the typed predicate without copying the source prose. For an exact sequence, explicitly use the appropriate kernel/image consequence rather than transferring injectivity, surjectivity, or isomorphism to an adjacent arrow.
6. Run an adjacent-arrow mutation: attach the predicate to each neighboring arrow in turn. The typed correspondence or a retained countermodel must reject every wrong attachment. Keep the displayed $\mathbb{Z}/2$ exact-sequence witness above as the minimal human-readable regression for the failure exposed above.
7. Separate source retrieval from consequence checking. A model that retrieves and then audits the same ambiguous sentence is not an independent route. Include a source-blind derivation of the local consequence and a separate primary-source/proof inspection for the imported premise.
8. Bind the same arrow identifier, signature, and predicate to any Lean theorem, executable schema, or certificate field. Treat a deep unformalized source theorem as an explicit typed premise; do not imply that an axiom or interface theorem verifies its truth.

Record `CLOSED` only when all applicable fields pass. `OPEN` means evidence remains to be supplied; `BLOCKED` means the source correspondence is ambiguous or false. The PDF checker retains the template, this gate, and its negative control in the human-readable artifact. That artifact gate does not inspect every future proof automatically; each claim packet must instantiate and review the record.

The deterministic `check-citation-edge-countermodel.py` gate exhausts the finite C_2 witness’s map tables, images, kernels, and isomorphism predicates. The companion mutation self-test script, `check-citation-edge-countermodel-self-test.py`, requires rejection of wrong-arrow binding, witness collapse, broken exactness, stale Markdown/LaTeX parity, and removal of the typed source-arrow field. Both run in the mathematical-workflow PDF gate, its `just` route, and CI. They reject the local inference schema; they do not interpret motivic homotopy or verify a PID theorem.

The orthogonal implementation check `PidCitationEdgeCountermodel.lean` states the same C_2 witness with Mathlib additive homomorphisms and proves the three image/kernel exactness equalities, right-arrow bijectivity and surjectivity, adjacent-arrow non-bijectivity and non-surjectivity, and middle-group nontriviality. Its pinned checker audits nine theorem declarations and their axiom inventories; its mutation gate rejects collapse of C_2 , replacement of the identity by zero, erasure of the kernel from exactness, and both false adjacent-arrow conclusions. This is useful implementation and kernel diversification against a defect in the Python checker’s custom finite-group semantics. It remains the same mathematical countermodel and the same logical route, not an independent counterexample. It does not formalize motivic homotopy, validate either cited source theorem, establish the source-to-Lean arrow correspondence, or prove anything about PID.





Cheap invariant probes

Before a long proof or expensive certificate run, derive low-cost invariants that every candidate must satisfy. Use them as early falsifiers and retain the smallest failure. Examples include:

- an integer-valued quantity must remain integral;
- a mathematical probability vector in an exact-real or rational representation must have nonnegative entries summing exactly to one; independently rounded binary64 entries instead need a proved rounding enclosure or a construction that enforces the represented normalization;
- a Möbius transform and its zeta inverse must reconstruct every coordinate;
- a symmetry claim must commute with the declared source permutation;
- an outward interval for a partial sum plus a proved two-sided remainder enclosure must combine into an enclosure of the total; in the special one-sided case, first prove $0 \leq r \leq R$ and require $L \leq s$ and $U \geq s + R$ for exact partial sum s ;
- a claimed covariance or concentration proxy must have the required sign and limiting behavior;
- a resource estimate must dominate the exact serialized object it is intended to bound.

An invariant can refute a route. Passing it does not prove the route. Whenever an invariant catches an error, add it to the correction ledger and convert it into a permanent negative control.

3.6.8 7. Use a blind holdout protocol

A proof and a holdout test answer different questions. A proof can establish a theorem under stated assumptions. A holdout test can estimate the behavior of an estimator or implementation on a defined benchmark distribution. Do not use one as a substitute for the other.

Call a benchmark blind only when the development and implementation roles cannot access the holdout rows, holdout seeds, target values, or unredacted adjudication output before the frozen analysis produces the complete first result table. Preregister every reported output. Record an access matrix for all roles and assets. Record who holds each sealed artifact and, as applicable, its binding-and-hiding commitment or encrypted-package digest. Never treat a raw digest of an enumerable secret as hiding. If these conditions do not hold, call the design a controlled holdout, not a blind holdout.

Use this protocol for estimator and pipeline claims:

1. Freeze the estimand, data-generating families, parameter grid, sample sizes, metrics, tolerances, failure rules, and analysis code.
2. Seal the generator, seed list, and targets. Before holdout access, publish an independently timestamped binding-and-hiding commitment: for low-entropy secrets, use a fresh high-entropy nonce retained under independent custody or bind an encrypted sealed package. A raw digest of an enumerable seed or target is not hiding.
3. Define the development, implementation, execution, and adjudication roles. Record all role overlaps and the access matrix.
4. Separate development, calibration, and holdout data. Do not tune on the holdout data.





5. Keep the holdout rows, seeds, target values, and unredacted result output unavailable to the development and implementation roles until the adjudicator records the complete first result table.
6. Include positive controls, negative controls, boundary cases, and known failure regimes.
7. Use repeated holdout draws for sampling claims. State confidence limits and preregistered acceptance criteria.
8. Report every planned cell. Do not remove a cell after a poor result.
9. Report effect errors, interval coverage, abstentions, failures, and resource limits.
10. Apply the stated multiplicity control when the decision uses many cells or hypotheses.
11. Record the first holdout result before a revision. A revision needs a new holdout version.
12. Publish the generator, manifest digest, adjudicator, and full result table after the blind phase.

The benchmark must define its scope. Synthetic Gaussian performance does not prove performance for mixed, singular, quantized, or dependent laws. A fixed benchmark does not prove a general theorem.

Minimum blind-benchmark commitment

The pre-access commitment must contain enough information to detect a change after result access. Record these fields:

Field	Required content
Benchmark ID	Stable versioned identifier
Claim IDs	Scoped empirical or calibration claims that the benchmark can accept or reject; never an unspecified universal or exact theorem
Analysis-plan digest	Digest of the metrics, tolerances, confidence limits, multiplicity rule, and acceptance rule
Code identity	Immutable source identity and environment or package-lock identity
Generator identity	Generator digest, parameter-grid digest, and sampling-law version
Sealed inputs	Binding-and-hiding commitment to the seed list or target bundle, or digest of an encrypted sealed package; record nonce/key custody separately
Role separation	People or systems that develop, implement, execute, and adjudicate, including all role overlaps
Access matrix	Permitted access for each role to code, rows, seeds, targets, keys, and result output
Commitment custody	Party that holds each sealed artifact, opening material, and binding-and-hiding commitment or encrypted-package digest, as applicable
Access rule	Event that permits target access and the party that records that event
Independent time evidence	Third-party timestamp or an independent adjudicator record
Failure rule	Treatment of crashes, non-finite values, abstentions, missing cells, and resource exhaustion





Field	Required content
Result identity	Digest of the complete first result table before any revision
Deviations	Every difference from the frozen plan

A local Git commit time alone is not independent time evidence. Do not store a secret seed, target, decryption key, or credential in the repository. The adjudicator must retain failed cells and must run the frozen analysis without manual result-dependent changes. If a confidentiality constraint prevents public release of a sealed input, publish a binding-and-hiding commitment and state who controls the sealed bytes and opening material.

This protocol reduces result-dependent tuning and selective reporting. It does not make a benchmark distribution representative, prove that an analytic target is correct, or replace a theorem.

3.6.9 8. Maintain a claim-to-evidence matrix

Use the matrix to prevent evidence substitution.

Claim class	Evidence that can support it	Evidence that does not complete it
PID definition and provenance	Defining paper, exact repository definition, and provenance marker	Similar name or similar output
Möbius reconstruction identity	Symbolic derivation and machine-checked finite algebra	Random numerical examples
Population functional theorem	Proof with explicit assumptions and counterexample audit	Passing implementation tests
Finite-alphabet consistency	Probability proof, stated mode of convergence, and formalized subclaims	One Monte Carlo curve
Numerical reference value	Symbolic value, rigorous interval, or high-precision value with error limit	Default floating-point agreement
Rust implementation conformance	Separately specified oracle with a recorded independence vector, mutation test, feature-path parity, and boundary tests	A paper citation
Scoped estimator calibration	Declared data-generating family, repeated preregistered holdout draws, confidence limits, and acceptance criteria	Reused development fixtures or one unqualified draw
Dependence-aware inference	Valid dependence assumptions plus block or permutation design checks	An independent-row test
Downstream suitability	Consumer-specific input contract and acceptance fixture	A generic PID example
Limitation or impossibility	Explicit counterexample or proved obstruction	Failure to find a proof

Each new or revised scientific claim must identify its applicable claim class or classes, evidence, and scope. A result can use more than one claim class. For example, a correct functional identity does not establish estimator calibration.





3.7 Application to shared-exclusions PID

The primary theoretical target in this project is the shared-exclusions PID line. Keep paper-defined quantities separate from project-defined diagnostics, wrappers, and workflows. `method-catalog.json` and its generated `METHODS.md` rendering are the authorities for method origin, code availability, and validation status. Their correlated checker enforces declared structure and route-specific semantic markers; neither the catalog nor checker is independent evidence that those scientific statements are true. The primary literature, typed mapping arguments, source/code correspondence, and counterexample audit remain separate obligations.

3.7.1 Exact claim packet for a new PID theorem

The packet must define:

- the ordered source variables and target;
- the construction identity: MGW finite-PMF, Schick-Poland general measure-theoretic, Ehrlich practical continuous, Williams–Beer $I_{\min}$, or another explicitly named functional;
- the nonempty source-index subsets, admissible antichains, and declared redundancy order;
- the construction-native local or target-outcome-specific information object and decomposition convention; for MGW, the pointwise informative, misinformative, and signed-net terms; where a construction has no such split, a typed `not_applicable` plus its native object;
- the averaging or conditioning law, or a typed `not_applicable` when the construction has neither;
- the logarithm base and units;
- the population support and any minimum-mass condition;
- the domain/codomain, sigma-algebras, dominating/reference measures, densities or RN derivatives, and measurement/preprocessing gauge;
- the row dependence model;
- the estimator and all fitted preprocessing;
- the requested conclusion and its convergence mode;
- the status of negative atoms;
- all cases where the method must abstain.

If the theorem concerns a plug-in estimator, distinguish the population functional from the empirical law and the implementation. If it concerns continuous data, state the support model. Do not infer full-dimensional absolute continuity from unique observed rows.

3.7.2 PID routing checkpoints

- Route by declared population law, source/target roles, source count, scientific goal, and gauge; never by storage dtype, observed ties/no ties, feature availability, or failure of another route.





- **SupportContract::AssumeRegularFullDimensional** is a conservative project-defined fail-closed runnable gate, not a verbatim statement of every cited paper’s analytic domain. It can reject paper examples with singular joint source support; rejection narrows code availability rather than refuting the paper functional.
- Standard KSG and continuous shared-exclusions estimator interpretation uses i.i.d. rows under its density/geometry premises. Eligibility requires unrounded rows from one fixed joint law, full-dimensional absolute continuity of every required marginal and joint law, finite mutual information, and declared boundary, smoothness/density, ambient-coordinate metric, and local-geometry conditions. A support declaration and observed uniqueness do not prove these premises. Subsampling or dependence-aware UQ does not itself prove estimator consistency for dependent rows; a separate theorem must name dependence coefficients and rates.
- KSG estimates mutual information using joint-neighborhood radii and marginal counts. Ehrlich redundancy uses a source-disjunction quasi-density/neighborhood construction, not a probability union law; the quasi-density need not integrate to one. Reuse of KSG-style neighbor counting as an implementation basis does not identify mutual information with Ehrlich shared-exclusions redundancy. Kraskov et al.’s higher-dimensional “redundancy” is multi-information/total correlation, not PID redundancy.
- O-information is a target-free Shannon scalar, not a PID atom or redundancy selector. With exactly two variables its formula is identically zero; redundancy- versus synergy-dominated sign interpretation begins only at three variables.
- The paper-defined target-conditioned average degrees $\bar{r}$ and $\bar{v}$ and pid-rs’ project-defined target-free Red°/Vul° ratios are different Shannon diagnostics. For $\bar{r}/\bar{v}$, bind one coherent target, law/sample, preprocessing, units, and positive denominator before forming the ratio. The target-free ratios are analogues for screening, not the published target-conditioned quantities. Neither family is a PID decomposition, atom, or redundancy selector, and no result transfers between them without a mapping theorem.
- **experimental::isx_heuristics** contains project-defined, formula-labelled comparison baselines. They do not estimate Ehrlich et al.’s paper-defined continuous shared-exclusions functional. Sharing KSG terms, an API shape, a wrapper alias, or a numerical coincidence does not supply a mapping.
- If a fitted quantizer artifact $\mathcal{Q}$ is random, distinguish the functional $F(P_q)$ conditional on one realized frozen artifact q , the artifact-averaged functional $E_{\mathcal{Q}}[F(P_{\mathcal{Q}})]$, and the artifact-marginalized-mixture functional $F(E_{\mathcal{Q}}[P_{\mathcal{Q}}])$. The second averages functional values; the third applies the functional to a mixture law. Nonlinearity supplies no equality. Stable fitted calls condition on the realized artifact and do not integrate over artifact randomness; hashes cannot prove row identity, observation nonoverlap, or fit/evaluation roles.
- Do not identify every equal-width implementation with the stable fitted-edge quantizer. pid-rs’ experimental same-sample route computes each column’s range on the same rows passed to the categorical estimator and selects bins by exact integer scaling of the computed binary64 fraction’s significand; it materializes no edge vector. At the ordinary boundary $[0, 1]$ with ten bins, binary64 0.3 maps to bin 2 by that rule but to bin 3 when tested against the stable route’s rounded edge vector. The two routes therefore define different represented categorical transforms at some boundaries. Bind the exact transform identity as well as **num_bins**; a shared “equal-width” label is not a mapping theorem.
- Sign rules are method-specific. Williams–Beer $I_{\min}$ has nonnegative exact-real finite-PMF atoms; a negative pid-rs output whose magnitude exceeds a separately justified numerical





error enclosure is a defect. Binary64 arithmetic alone supplies no universal tolerance. MGW informative and misinformative component atoms are separately nonnegative in the exact-real theorem, while their signed nets may be negative. Ehrlich continuous raw PID atom estimates may be signed. Population mutual information is nonnegative, but a finite-sample raw KSG estimate may be negative and has its own scalar **NegativeHandling** policy. Componentwise clamping of a negative estimated input or atom inside a PID composition can break raw identities unless the other atoms are changed. Ehrlich's suggested atom rebalancing can retain consistency equations (physical PDF page 18), but it is a transformed decomposition rather than the raw atoms. No pid-rs continuous-PID API implements that atom rebalancing; KSG scalar clamping is a separate MI reporting policy, not a PID transform.

- “Mixed-dimensional PID3” means continuous antichain branches with different source-block dimensions, not mixed discrete/continuous support. Ehrlich Appendix B's logical-statement quasi-density and Appendix K's research-sketch mixed-system estimation ansatz do not supply a generally calibrated pid-rs mixed estimator.
- Barà et al.'s discrete-target/continuous-source route uses Williams–Beer minimum-specific-information, not shared exclusions. Reversing target/source support roles or sharing natural-log units does not preserve the estimator.
- A fitted quantizer or PCA/PLS projection defines transformed variables. For a claim conditional on a frozen artifact, a held-out evaluation, or population inference under a fixed transform, train using only declared disjoint fit information, freeze and bind the artifact to evaluation, and type which variables informed fitting. Target-supervised fitting on disjoint training targets is permitted when declared. An explicitly same-sample adapter instead defines a data-dependent transform and downstream empirical estimator. Under the current pid-rs evidence and status, it is exposed only for its declared exploratory or descriptive target; it does not inherit that fixed-transform or held-out theorem, and population inference would require a separate theorem for the joint fitting-and-evaluation procedure. Evaluation-target or same-row confirmatory-target fitting is leakage relative to a held-out claim. No general theorem preserves PID atoms through PCA/PLS or makes fixed bins converge to the unquantized continuous functional.

3.7.3 Complementary PID audit families

For each new theorem, use at least five genuinely failure-diverse applicable audit families. A shared oracle, imported lemma, generated table, implementation, or formalization seam counts once at its common cut even when several suite names exercise it. If a listed family is inapplicable, record a typed reason; inapplicability is not replacement evidence and does not reduce the required 20-lens applicability matrix for a major claim.

- **Combinatorial:** antichain order, Möbius inversion, atom reconstruction, and source symmetry.
- **Analytic:** continuity, support boundaries, limiting arguments, and perturbation bounds.
- **Probabilistic:** concentration, dependence colorings, mixing assumptions, and resampling.
- **Computational:** exhaustive small alphabets, exact count tables, and counterexample search.
- **Formal:** Lean or SMT statements for finite algebra and deterministic inequalities.





- **Statistical:** coverage, power, null calibration, selection effects, and blind holdout behavior.

These families can support each other. They are not interchangeable. A formal lattice identity does not prove a concentration theorem. A concentration theorem does not prove that a continuous kNN estimator targets the intended functional.

3.7.4 Required counterexample search

For each new PID claim, test the relevant cases:

- independent sources and target;
- copied sources;
- XOR and other synergy gates;
- duplicate or deterministic sources;
- zero-probability and near-zero-probability cells;
- support deletion and support creation;
- singular and mixed-dimensional continuous laws;
- exact ties and quantized observations;
- row dependence and a false dependence partition;
- cancellation in reconstructed atoms;
- source permutation and bijective target-state relabeling; separately, noninjective target coarse-graining as an estimand-changing map;
- admissible state/event identifications and incidental overlaps, including collapsed source or target categories and duplicate coordinates;
- serial and parallel implementation paths.

Keep a counterexample when it breaks a proposed unconditional claim. State the corrected assumption next to the retained example.

3.7.5 Formal and software evidence

Formalize the exact theorem statement before large certificate generation. Retain a reviewed prose-to-formal correspondence table for every domain, codomain, quantifier, order relation, event, premise, and conclusion. Record the exact proof-assistant/kernel/library toolchain, invoke `--trust=0` or the closest available minimal-trust audit, inventory every theorem's axioms, and name all unformalized bridges. Kernel acceptance establishes that the exact recorded checker/toolchain accepted the encoded term. Deductive proof credit is conditional on the encoded calculus and the checker/kernel implementation being sound and on no applicable soundness exploit. Record and review known soundness advisories against the exact version, including whether a rejected version must be replayed on a fixed release. Even then, acceptance does not prove that the encoding is the intended prose/scientific object.

For a Möbius claim, formalize the declared finite antichain carrier and its actual partial order. Do not substitute an untyped matrix, category-theoretic story, or infinite-poset inversion without





a separate correspondence theorem and, where applicable, local-finiteness/convergence premises. For finite domains, use exact enumeration or certified intervals when possible. Then replay the same cases through the Rust API and any claimed Python wrapper.

Use mutation tests for proof and evidence scripts. Require malformed fixtures, wrong theorem bindings, weakened premises, altered source counts/orders, missing branches, and false conclusions to fail closed. Run checkers under normal Python and Python with optimization enabled (the `-O` option); assertions must not be the acceptance mechanism. Where two solvers or implementations are claimed, state their functional/epistemic/institutional independence vector and ensure a mechanistically separate checker recomputes the obligation rather than merely parsing the producer's claimed answer. A different model lineage may improve epistemic diversity but is not institutional independence; institutional custody requires separately controlled authority, access, and refusal capability. Run every API, feature, backend, compiler mode, wrapper, and build mode named by the claim. None of these checks proves generic proof-kernel soundness, compiler correctness, or platform-independent transcendental arithmetic.

3.7.6 Statistical and ecosystem evidence

Define separate benchmark strata for categorical MGW SxPID, fitted quantized SxPID, discrete $I_{\min}$, continuous KSG mutual information, Ehrlich continuous redundancy, continuous PID2, incomplete PID3 availability diagnostics, research mixed-dimension PID3 coordinates/branches, project-defined continuous heuristics, Shannon invariant families, uncertainty procedures, and Rust/Python wrapper parity. A composed report belongs to a further stratum that names every input estimand and mapping; it does not merge their evidence. Each stratum needs its own analytic target or certified numerical reference. If a stratum has neither, label it diagnostic. Each stratum also needs its own tolerance and abstention policy. Each mapping or routing benchmark must include at least one neighboring non-target method or estimand that it must reject, so numerical coincidence cannot serve as a mapping theorem.

For Prisoma, Haldir, Galadriel, and Crebain, record a consumer contract. The contract must identify the required estimand, data support, dependence model, scale, uncertainty output, resource limit, and failure behavior. Do not claim consumer readiness until a versioned acceptance suite checks every contract field. One fixture supports only its declared case. Do not infer consumer readiness from general unit tests.

3.8 Full semantic closure

A model, solver, proof assistant, generator, or test must check the complete object in the claim packet. For a PID claim, this can include:

- every nonempty antichain for the declared source count;
- every event union and target intersection in the definition;
- every supported realization;
- every source permutation named by a symmetry claim;
- every empirical count table in a declared exhaustive domain;
- informative, misinformative, net, tie, and abstention branches;
- every API, feature, serial or parallel path, and specialized or general path named by the claim;





- every fitted transform and split named by a consumer contract.

If complete enumeration is impossible, supply a proved reduction or a complete separation oracle that terminates on the declared domain and returns either a violator or a checkable no-violation certificate. A semidecision search that may run forever in the no-violator case remains a falsifier search. Random examples do not establish a universal equivalence statement. State the exact domain and resource bound of every finite search or oracle call.

3.9 PID exceptional-case checklist

Boundary analysis is a separate obligation. For each applicable item, state whether the theorem extends, changes statement, or requires abstention:

- zero or near-zero supported mass;
- support deletion or creation;
- an event denominator that approaches zero;
- exact or near $I_{\min}$ ties;
- informative and misinformative cancellation or an atom near zero;
- duplicate, copied, deterministic, or constant variables;
- an invalid dependence coloring;
- a transform fitted on evaluation rows;
- drift, adaptive selection, or repeated use;
- a zero KSG radius or nonunique neighbor shell;
- unequal intrinsic dimensions or incompatible reference measures;
- a mixed-dimensional or singular law;
- integer, allocation, recursion, and cancellation boundaries;
- platform-dependent transcendental arithmetic.

A derivation that divides away a boundary must keep a separate obligation for that boundary.

Determinism requires typed support analysis. For Euclidean (hence standard-Borel) variables, if $Y = f(X)$ for Borel-measurable f and P_Y is non-atomic, the measurable graph is $(P_X \otimes P_Y)$ -null but has joint probability one, so KL mutual information is infinite. The blanket statement “deterministic maps have infinite MI” is false: a thresholded Bernoulli output can have finite $I(X; Y) = H(Y)$, and a constant output has zero MI. Those atomic-output examples still do not satisfy an ordinary full-dimensional continuous KSG contract.





3.10 Layered assurance and go/no-go gates

Use each applicable layer. No layer substitutes for another.

Gate	Required closure	Claim disposition while open	Prohibited wording while open
G0 Claim identity	Frozen packet, sources, non-solutions	blocked	Claim scope is final
G1 Conventions and premises	Convention and assumption map	blocked	Premises are discharged
G2 Mathematical core	Proof or counterexample and boundary audit	active or blocked	The theorem or disproof is complete
G3 Formal semantics	Actual objects and implication in a proof checker	active or blocked	Formally verified
G4 Certified numerics	Rigorous enclosure and unresolved semantics	active or blocked	Certified sign or tie
G5 Executable conformance	Refinement or bounded complete equivalence	active or blocked	Verified implementation
G6 Estimator calibration	Preregistered scoped calibration	active or blocked	Calibrated beyond the tested scope
G7 Consumer qualification	Versioned contract and acceptance suite	blocked	Consumer-ready or authority-grade
G8 Release archive	Reproducible builds, hashes, and first-result record	blocked	Final release assurance

An inapplicable gate needs a written reason. A release statement must name the closed and open layers. **falsified** is terminal only after an accepted counterexample closes a negative result; it is not an “open” G2 state.

For a major claim, keep a revision-preserving directory:

```
claims/<CLAIM-ID>/
  claim-v1.md
  conventions.md
  obligations.md
  routes.md
  failures/
  certificates/
  formal/
  implementation/
  benchmark/
  audit.md
  evidence-matrix.md
  decision.md
```

Do not add empty placeholders. Create a file when it has evidence or a required open obligation. Do not overwrite old claim revisions, failed routes, certificate inputs, or first-result records.

3.11 Acceptance rule

Maintain an append-only accepted-result evidence list and a separate claim disposition. Each record binds exactly one evidence class plus its scope, assumptions, exact artifact identities, adjudication, and invalidation/supersession state. Render the empty list literally as “no accepted evidence”; later records never erase contradictory or invalidated records. Allowed classes are:

- theorem proved under stated assumptions;





- machine-checked finite obligation;
- certified numerical bound;
- preregistered empirical result;
- counterexample;
- diagnostic observation.

The disposition is exactly one of proposed, active, blocked, falsified, or complete. **complete** requires every applicable obligation to be closed; closing a finite or machine-checked sub-obligation does not close its parent. A counterexample may complete a disproof, never the original proof claim.

Contradictory accepted-looking evidence forces **blocked**. Retain both artifacts, identify the smallest disputed obligation, and resolve it before either route can close the claim.

Never mutate a record or change a disposition without new evidence. Append an invalidation or supersession event and link every artifact to the claim ID. This ledger exposes gaps; it does not prove the claim.

